IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received February 20, 2021, accepted March 7, 2021, date of publication June 3, 2021, date of current version September 9, 2021.

Digital Object Identifier 10.1109/ACCESS.2021.3073480

Improved Initialization Method for Metaheuristic Algorithms: A Novel Search Space View

QIAN LI, YIGUANG BAI, AND WEIFENG GAO
School of Mathematics and Statistics, Xidian University, Xi'an 710071, China

Corresponding author: Qian Li (qli_5@stu.xidian.edu.cn)

This work was supported in part by the National Natural Science Foundation of China under Grant 61877046 and Grant 61877047, in part by the Fundamental Research Funds for the Central Universities, and in part by the Innovation Fund of Xidian University.

ABSTRACT As an essential step of metaheuristic optimizers, initialization seriously affects the convergence speed and solution accuracy. The main motivation of the state-of-the-art initialization method is to generate a small initial population to cover the search space as much as possible uniformly. However, these approaches have suffered from the curse of dimensionality, high computational cost, and sensitivity to parameters, which ultimately reduce the algorithm's convergence speed. In this paper, a new initialization technique named diagonal linear uniform initialization (DLU) is proposed, which follows a novel search view, i.e., adopting the diagonal subspace sampling instead of the whole space. By considering the algorithm's update mechanism, the improved sampling method dramatically improves the convergence speed and solution accuracy of metaheuristic algorithms. Compared with the other eight widely used initialization strategies, the differential evolution (DE) algorithm with DLU obtains the best performance in search accuracy and convergence speed. In the extension experiments, results show that the DLU is still effective for three swarm-based algorithms: particle swarm optimization (PSO), cuckoo search (CS), and artificial bee colony (ABC). Especially for the multi-objective problem, the DLU still demonstrates its powerful performance compared with other strategies.

INDEX TERMS Optimization, metaheuristic algorithm, initialization, evolutionary algorithm, swarm intelligence algorithm, multi-objective optimization.

I. INTRODUCTION

Compared with traditional optimization such as Newton's method and Gradient Descent, metaheuristic algorithms [1], [2] are sophisticated global optimizers with high robustness and wide applicability. These new successful optimizers can effectively deal with complex problems that are difficult to be solved by traditional optimization. In fact, metaheuristic optimizers have been successfully applied in engineering design [3]–[5], decision management [6], the Internet of things [7], complex network [8], job scheduling [9]–[11], genetic engineering [12], [13], biomedical [14], resource allocation and other fields.

In the past decade, the study of metaheuristic algorithms is still a hot research topic. Researchers were devoted to studying more effective exploitation and exploration mechanisms to improve the performance (such as convergence speed, optimization accuracy, etc.) of metaheuristic optimizers. Hence, some new metaheuristic strategies have been proposed, such as the grey wolf optimization (GWO) [15], [16], whale optimization algorithm (WOA) [17], naked mole-rat algorithm [18] and animal migration optimization [19], etc. But a crucial algorithm step: initialization, is often ignored.

As a population-based stochastic search technique, the metaheuristic algorithm is deeply affected by the initial population. One suitable initial population can make the algorithm converge faster and find more accurate solutions [20], [21]. It is obvious that if the initial population is unfortunately located in the local optimal region, the subsequent search is not easy to jump out. This is especially true for multi-modal and high-dimensional problems.

Considering the impact of the initial population, lots of initialization strategies are proposed recently. Theoretically, existing initialization methods can be roughly divided into two categories:

The associate editor coordinating the review of this manuscript and approving it for publication was Hisao Ishibuchi.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

A. WITHOUT PRIOR KNOWLEDGE

In most cases, we have little/no prior information on the problem to be solved. Fortunately, as a black-box optimization, metaheuristic algorithms can overcome this difficulty by simple initialization without prior knowledge. Based on a recent research [22], this category can be classified into the following two groups: (1) randomness, e.g., Machine-generated pseudo-random numbers [23]–[26], Uniform experimental design [27]–[29]; (2) based on special methods, such as Chaotic techniques [30], [31] which can extract the search space information to increase the population diversity; Opposition-Based learning method [32], which has been utilized to accelerate the convergence rate; Sequence-based deterministic initialization [33] and hybrid techniques [21], [34], etc.

However, these approaches have some drawbacks. Most of these initialization strategies are based on a guiding ideology: uniformly cover the search space as much as possible. The problem is that this motivation often fails in large-scale spaces, i.e., many initialization methods suffer from the curse of dimensionality [35]. Furthermore, some of these approaches are sensitive to initial conditions and their parameter settings, such as Chaotic techniques [36] and Orthogonal design [37]. Hybrid techniques and multi-step techniques [38] suffer from three main limitations: high computational costs, greedy selection mechanisms and easy to lose hidden information in the initial stage [22]. More importantly, all these methods only pay attention to the uniformity of the current population distribution, without considering the characteristics of the iteration (update and select mechanism) of the algorithm itself.

B. WITH PRIOR KNOWLEDGE

If some prior knowledge of related problems can be captured, it will be used to guide the distribution of the population. For example, Li *et al.* [39] applied knowledge-based initialization combined with the genetic algorithm to solve the traveling salesman problem. Chang *et al.* [40] suggested that the heuristic factors can be used in population initialization to improve the quality of an improved artificial bee colony algorithm for the dynamic weapon target assignment problems. Li *et al.* [8] used the information of nodes to initialize population in solving the network disintegrates problems.

The issue is that these approaches may only be applicable to specific types of problems or algorithms. Besides, these initializations are highly dependent on the experience and prior knowledge of the user. In fact, there is very little mathematical guidance on initialization methods for practical problems.

Motivated by the above-mentioned research gap, a new initialization strategy called diagonal linear uniform initialization (DLU) is proposed in this paper. The entire search space is evenly divided into many unit subspaces, and we select some special points in the diagonal subspace as the initial population. DLU is a simple initialization method but very effective in metaheuristic optimizers. The proposed technique is not to make the population distribute as uniformly as possible in the initial stage, but to make the population carry the information of various dimensions evenly. The iterative characteristics of metaheuristic algorithms are taken into account. Note that this method is different from Latin hypercube sampling [41] and diagonally uniform initialization; details will be clarified later. The experimental results show that the proposed DLU is more effective than other commonly used initialization methods and is also suitable for most metaheuristic algorithms. Moreover, DLU combines with a widely used multi-objective algorithm framework: the multi-objective evolutionary algorithm based on decomposition (MOEA/D) [42] for solving six well-known benchmark problems to prove that DLU can improve the performance of multi-objective algorithms.

The remaining of this paper is organized as follows: Section II briefly introduced some widely used initialization methods, including Chaotic initialization, Opposition-based learning, Diagonal uniform distribution, Latin hypercube sampling, and Sequence-based deterministic initialization. Some details of the diagonal linear uniform initialization (DLU) are presented in Section III. Experimental results are presented in Section IV, together with the comparison of various initialization approaches on different metaheuristic optimizers, we also test the proposed DLU with multi-objective benchmark problems. Then, Section V discussed additional experiments on some problems from CEC2014 and CEC2017, and analyzed the diversity of the population in the iterative process. Finally, the conclusion is drawn in Section VI.

II. BACKGROUND

This section will briefly introduce some popular initialization methods: Chaotic initialization, Opposition-based learning, Diagonal uniform distribution, Latin hypercube sampling, and Sequence-based deterministic initialization. At the same time, the strengths and weaknesses of these approaches are discussed and analyzed in detail.

A. CHAOTIC INITIALIZATION

The nature of chaos is apparently random and unpredictable, and chaotic sequences have been proven easy and fast to generate and store [43]. Therefore, chaotic sequences have been adopted instead of random sequences recently. Evidence shows that the use of chaotic sequences in the artificial bee colony algorithm can effectively prevent the algorithm from falling into local minima [31]. There are many kinds of chaotic maps [44], [45], such as Chebyshev map, circle map, iterative map, logistic map, tent map, etc. We choose two chaotic maps as examples:

Gauss map:

$$cm_{k+1} = \begin{cases} 0, & cm_k = 0 \\ \dfrac{1}{cm_k} - \left\lfloor \dfrac{1}{cm_k} \right\rfloor, & cm_k \in (0, 1) \end{cases} \tag{1}$$

where $\lfloor x \rfloor$ represents the largest integer not exceeding x.

Sinus map:

$$cm_{k+1} = 2.3(cm_k)^{2\sin(\pi cm_k)} \tag{2}$$

The mapped variables can be randomly and irregularly distributed in search space. Hence, the chaotic system is used to generate the initial population, which is considered to replace random initialization. The initial population was generated as described in Alg. 1.

Algorithm 1 Chaotic Initialization

Input: Set the maximum number of chaotic iteration $K = 300$, the population size N, and $x_{i,j}$ is the j-th dimension of the i-th individual.
for $i = 1$ to N **do**
  for $j = 1$ to D **do**
    Randomly initialize variable $cm_{0,j} \in [0, 1)$, set the iteration counter $k = 0$
    for $k = 1$ to $K - 1$ **do**
      $cm_{k+1,j} = f(cm_{k,j}), f$ is the chaotic mapping.
    end for
    $x_{i,j} = x_{i,j_{min}} + cm_{K,j}(x_{i,j_{max}} - x_{i,j_{min}})$, $x_{i,j_{min}}$ indicates the lower bounds of j-th dimension of the i-th individual
  end for
end for
return $X = (x_{i,j})_{N\times D}$ as a vector.

As shown in Eq. 1 and Eq. 2, the process of generating chaotic sequences involves several functions (chaotic mapping) and several parameters (initial conditions). Therefore, it inevitably has a very sensitive dependence on its initial conditions and parameters [46].

B. OPPOSITION-BASED LEARNING

The opposition-based learning (OBL) has been applied to accelerate reinforcement learning and backpropagation learning in neural networks. Rahnamayan *et al.* utilized the concept of OBL to improve the convergence rate of differential evolution algorithms. They believe that the chances of approaching better solutions may be improved by checking the opposite solutions simultaneously [32].

The opposite solution is defined as:

$$\breve{x}_{i,j} = x_{u,j} + x_{l,j} - x_{i,j}, \tag{3}$$

where $x_{i,j}$ representing the j-th dimension of the i-th individual, $x_{u,j}$ and $x_{l,j}$ are indicate the upper and lower bounds of j-th dimension, respectively. Hence, $\breve{x}_{i,j}$ denote the opposite solution of $x_{i,j}$.

The opposition-based learning initialization is shown in Alg. 2. However, estimating the fitness value is usually considered as a time-consuming step in algorithms. It is easy to see that N individuals with better fitness value are selected as the initial population after $2 \times N$ fitness evaluations in the opposition-based learning method. Double times of fitness evaluations double the computational cost.

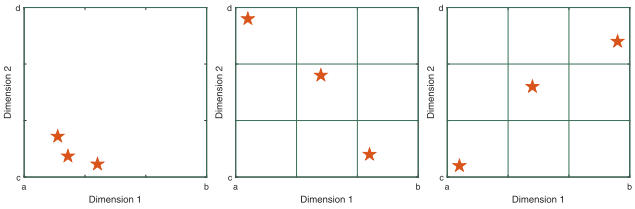
Algorithm 2 Opposition-Based Learning Method

Input: the population size N, and $x_{i,j}$ is the j-th dimension of the i-th individual
Initialize the population $P = (x_{i,j})_{N\times D}$ randomly.
for $i = 1$ to N **do**
  for $j = 1$ to D **do**
    $\breve{x}_{i,j} = x_{u,j} + x_{l,j} - x_{i,j}$
  end for
end for
return $\breve{P} = (\breve{x}_{i,j})_{N\times D}$ opposite population.
Select N fittest individuals from $P \cup \breve{P}$ as the initial random population.

C. DIAGONAL UNIFORM DISTRIBUTION

Choosing an initial point from each unit subspace as the initial population can avoid losing some information at the beginning. However, this will cause a large number of initial individuals, which will greatly reduce the convergence speed of the algorithm. As a compromise, Chou *et al.* deem that the diagonal linear subspaces of the search space contain much valuable information and proposed an initialization scheme [47]: diagonal uniform distribution (DUD) for genetic algorithm (GA).

Firstly, the search space is divided into several subspaces through the grid. To be specific, each dimension is divided equally into N(N is the population size) parts. Note that the search space is eventually divided into N^D subspaces. Then, generating population individuals in the diagonal subspace. For example, suppose the space is two-dimension and the population size is three. The diagonal uniform distribution initialization is shown in Fig. 1(b) or Fig. 1(c).



(a) Random distribution. (b) Leading diagonal uniform distribution. (c) Paradiagonal uniform distribution.

FIGURE 1. Diagram of sampling point distribution in two-dimensional space: the orange ★ indicates the selected.

Compared with the stochastic method, as shown in Fig. 1(a), the initial points generated by DUD have a broader coverage, which effectively avoids the population excessive aggregation. Experiments in [47] have also shown that DUD is feasible and effective. However, this effect will diminish as the dimensionality increases.

D. LATIN HYPERCUBE SAMPLING

Latin hypercube sampling (LHS) is a spatial filling mechanism, which utilizes ancillary variables to ensure each of

the variables to be represented in a fully stratified feature space [48]. Latin hypercube sampling is a hierarchical sampling technology, which based on optimization. The later selected points are affected by the previous points. The diagonal uniform initialization can be regarded as a particular case of Latin cube sampling.

LHS requires that each hyperplane perpendicular to the axis contains at most one sample. As the example mentioned above, the dimension of the search space is two, and three sampling points are needed. For LHS, there are six situations. Except for the two diagonal subspace distributions in section II-C, there are four other possible situations, as shown in Fig. 2.

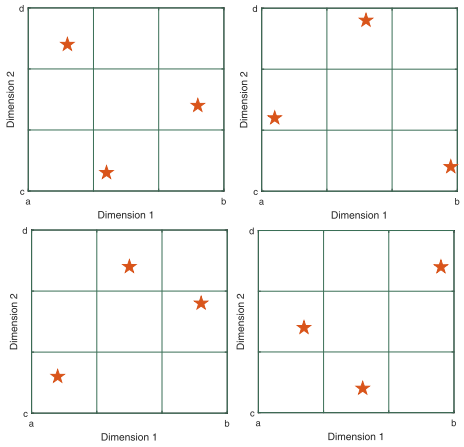


FIGURE 2. Four possible distribution scenarios of Latin hypercube sampling in 2D space [49]:the orange ★ indicates the selected point.

Compared with the simple random sampling of the Monte Carlo simulation method, the Latin hypercube sampling method has a higher spatial coverage rate, and its essence is that the standard deviation of samples is smaller. Latin hypercube sampling combines the desirable features of many random and stratified sampling, so it produces more stable analysis results than random sampling [50]. It has become a widely used initialization method in recent years. But LHS suffers from the curse of dimensionality. Consider a 10-dimensional problem and assume that the population size is 100. Then the search space will be divided into 100^{10} subspaces, and sampling 100 points in so many subspaces. In fact, the population distribution is very sparse. LHS performs well in low-dimensional space but becomes inefficient as the dimensionality increases. This significantly limits the performance of LHS.

E. SEQUENCE-BASED DETERMINISTIC INITIALIZATION

Elsayed *et al.* [33] proposed an initialization method called the sequence-based deterministic initialization (SDI). Let $\vec{x}_u$ and $\vec{x}_l$ are the upper and lower bound vectors of the search domain, respectively. The interval ($\vec{I}$) can be determined as:

$$\vec{I} = \frac{(\vec{x}_u - \vec{x}_l)}{q-1}. \tag{4}$$

Then the vector of initial individual is generated as follows:

$$\vec{P}_k = \begin{cases} \vec{x}_u, & k = 1 \\ \vec{P}_{k-1} + \vec{I}, & 2 \le k \le q-1 \\ \vec{x}_l, & k = q \end{cases} \tag{5}$$

By Eq. 5, we can obtain q vectors. Subsequently, for every $\vec{P}_k$ (such as $\vec{P}_{k_i}$), choose one dimension (for example, the j-th dimension) at a time. Then change the value of this dimension to all possible value $P_{k,j}$, ($k = 1, 2, \cdots, k_i - 1, k_i + 1, \cdots, q$). It is worth noting that only one dimension changes at a time. In this way, $q \times (D \times (q-1) + 1)$ individuals can be generated, and all these individuals form the initial population.

The number of individuals that can be generated by SDI is much smaller than the uniform experimental design [51]. However, it is obvious that the number of initial population cannot be arbitrarily selected (Integers such as 100 or 500 cannot be implemented). When the value of q is determined, the total number of population is invariant. For a 30D problem, when each dimension is divided into 10 equal parts, i.e. $q = 11$, the number of individuals is 3,311, and for a 100-dimensional problem, 11,011 individuals need to be selected. If the grid is relatively dense (each dimension is divided into more parts), the number of the initial population will be surprisingly large, and the calculation cost is very high.

III. DIAGONAL LINEAR UNIFORM INITIALIZATION

A. DESCRIPTION OF SAMPLING TECHNIQUE

For metaheuristic optimizers, the selection of the initial population can significantly affect the convergence speed [52]. Ideally, the initial population should be not only easy to obtain (low computational costs), but also effectively improve the performance of the algorithm. However, the existing approaches have many shortcomings. In fact, we lack more effective initialization strategies and useful guidance to overcome complex problems. More importantly, in the initialization step, scholars had focused on the uniformity and diversity of the population distribution without considering the influence of the update mechanism of the algorithm. Therefore, it is necessary to explore more reasonable initialization techniques to improve the search capability of the algorithm.

Based on the above considerations, we proposed a new initialization strategy, called diagonal linear uniform initialization (DLU). Firstly, each dimension of the space is divided into $N-1$ equal parts, and then some vertices of the diagonal subspace are selected. As shown in Fig. 3, the proposed method can also be regarded as taking some uniform points on the 'diagonal' of the space, and the distance between any two adjacent points is $\frac{(\vec{x}_u - \vec{x}_l)}{N-1}$.

Taking a 3-dimensional space as an example, the upper and lower bound vectors of the search domain are $(-2, -2, -2)$ and $(2, 2, 2)$, respectively. Let us suppose that five initial individuals are needed. According to diagonal linear uniform

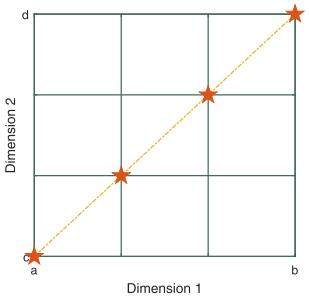


FIGURE 3. The 2D diagram of diagonal linear uniform initialization: the orange ★ indicates the selected point.

initialization, each dimension is divided equally into four parts and the five initial points selected are $(-2, -2, -2)$, $(-1, -1, -1)$, $(0, 0, 0)$, $(1, 1, 1)$, $(2, 2, 2)$, which are represented by orange ★ in Fig. 4.

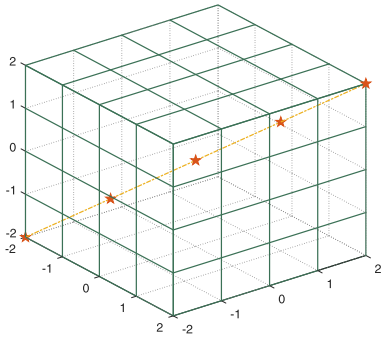


FIGURE 4. The 3D diagram of diagonal linear uniform initialization (DLU).

Algorithm 3 Diagonal Linear Uniform Initialization

Input: the range $[x_{i_\min}, x_{i_\max}]$ for each dimension x_i. the population size N, the dimension of problem D.
$X_{init} = zeros(N, D)$ // allocate memory space
for $j = 1, 2, \ldots, N$ **do**
  for $i = 2, \ldots, D-1$ **do**
    $X_{init}(j, 1) = x_{i_\min}$,
    $dx = \frac{x_{i_\max} - x_{i_\min}}{N-1}$ // length of each interval in i-th dimension
    $X_{init}(j, i) = x_{i_\min} + (i-1) * dx$
    $X_{init}(j, D) = x_{i_\max}$
  end for
end for
return X_{init} as a vector.

Diagonal linear uniform initialization is simple and easy to implement. The pseudo code of DLU is described in Alg. 3. The most important thing is that its performance does not diminish with increasing dimensionality, and it performs well in many problems (e.g., multi-objective, multi-modal, etc.). Details will be given in the experimental section.

B. ANALYSIS AND COMPARISON

Since population-based algorithms rely on random guessing during initialization, there are chances that a specific search may restrict the population to certain parts of the space. This can lead to premature convergence. The proposed DLU can avoid this situation well and improve the search capability of the algorithm.

Taking a typical multi-modal function as an example: the Rastriginas function has many local optimums, which makes algorithms easy to fall into the local optimums, as shown in Fig. 5. To validate DLU's priority, we choose this function to investigate the possible impacts of different initializations on the algorithm (such as PSO). In order to visualize the distribution of the population, the 2D spatial situation is first discussed. The search space is $[-5, 5] \times [-5, 5]$. The magenta hollow star is used to represent the optimal value $(0, 0)$, and the initial individuals are represented by blue dots. As a comparison, Fig. 6 shows the initial population generated by the Random method, in which the best solution in the current population is represented by red ★. It can be seen that after 10 iterations of the PSO algorithm, the distribution state of the population is shown in Fig. 6(b). The best solution found by the algorithm is still far from the real optimum, which means that the algorithm has fallen into the local optimum. However, the position state of the initial population generated by DLU is shown in Fig. 7(a). After 10 iterations of PSO, the positions of the overall individuals are shown in Fig. 7(b), they are clustered near the optimal solution, and the current best solution is consistent with the global optimum. This means that DLU is more conducive to the algorithm to find the optimal solution.

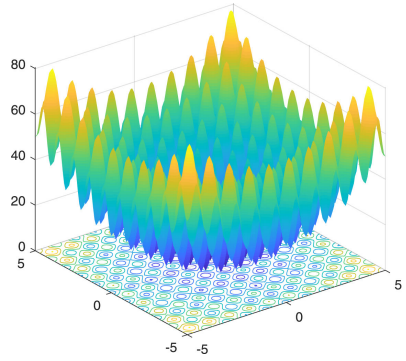
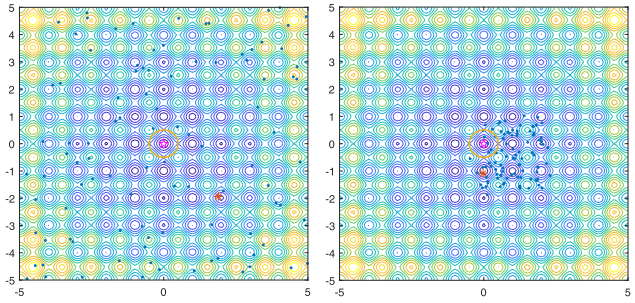


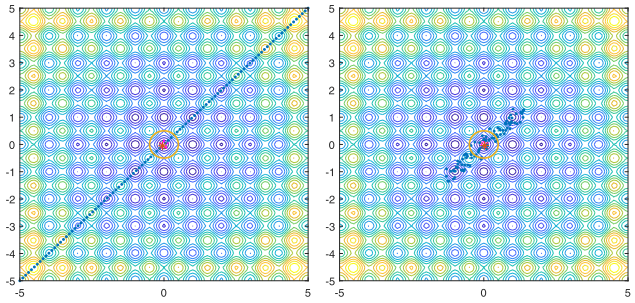
FIGURE 5. The 2D landscape of Rastriginas Function.

In order to strictly compare these two approaches, we also calculate the number of individuals in the neighborhood of the optimal solution. In this example, centered on the global optimum, a circular area with a radius of 0.5 is the neighborhood of the optimal solution, which is represented by an orange circle in the figures. For Random method, 13 individuals are located near the optimal solution after 10 iterations. DLU causes 70 individuals to move to the neighborhood of the optimal solution. This means that DLU can improve the performance of the algorithm.



(a) Initial distribution. (b) After 10 iterations of PSO.

FIGURE 6. The initial population obeys random distribution.



(a) Initial distribution. (b) After 10 iterations of PSO.

FIGURE 7. The initial population distribution with DLU initialization.

In fact, our DLU still has obvious advantages in high-dimensional space. Note that it is difficult to give a visualization diagram, so we chose statistical data in the analysis. Assume that the Rastriginas function in a 30-dimensional space. After 10 iterations by PSO, none of the population generated by the Random method is located in the spherical neighborhood of the optimal solution (the optimal point is the center of the sphere, and 0.5 is the radius), while DLU makes 24 individuals move to the vicinity of the optimal solution. Obviously, the initial population generated by DLU is easier to find the optimal solution. It can be considered that the DLU method improves the search capability of the algorithm.

The proposed DLU is simple but very efficient for metaheuristic algorithms. There are two main reasons:

Firstly, the initial population generated by this strategy carries information of each dimension at equal intervals. It can be understood that these initial populations provide a wealth of 'gene' resources for subsequent crossover and mutations. It can effectively avoid excessive aggregation of information in some dimensions, especially in large-scale problems.

Secondly, the previous methods pay more attention to the diversity of the initial population, and ignores the update mechanism of algorithms. DLU considers the influence of iterative mechanism in the algorithm process. In fact, most meta-heuristic optimizers have a linear iteration mechanism. For example, the mutation strategies of DE:

$$v_{i,j} = x_{r_1,j} + F \cdot (x_{r_2,j} - x_{r_3,j}). \tag{6}$$

where $v_{i,j}$ is the j-th dimension of the i-th individual after the mutation, F is a parameter for mutation strength. Here, $x_{r_1,j}$, $x_{r_2,j}$, $x_{r_3,j}$ represent the j-th dimension of 3 different individuals, which are selected randomly from the current population.

The update equations of PSO:

$$v_i{}^{t+1} = w \cdot v_i{}^t + c_1 r_1{}^t (p_i{}^t - x_i{}^t) + c_2 r_2{}^t (p_g{}^t - x_i{}^t) \tag{7}$$

$$x_i{}^{t+1} = x_i{}^t + v_i{}^{t+1}. \tag{8}$$

where $v_i{}^t$ and $x_i{}^t$ represent the velocity and position of particle i at t-th iteration. c_1 and c_2 are the learning parameters, while r_1 and r_2 are two random values. $p_i{}^t$ is the best solution of i-th individual, and $p_g{}^t$ is the best solution of the current population.

The global search and local search of CS:

$$x_i^{t+1} = x_i^t + \alpha \oplus L(s, \lambda), \tag{9}$$

$$x_i^{t+1} = x_i^t + \alpha s \otimes H(p_a - \varepsilon) \otimes (x_j^t - x_k^t) \tag{10}$$

where α is a controlling factor, s is the step size, and ε drawn from a uniform distribution. Here, x_j^t and x_k^t are two random individuals at t-th iteration. L represent a Lévy distribution and $H(u)$ is the Heaviside function [53].

Although the update strategies are different, almost all metaheuristic optimizers use linear iteration to find the best solutions. These linear iterations combined with the uniform interval information in each dimension can generate coordinates for any position in space. As long as the coefficient of linear iteration is not equal to 0, the initial population will be clustered in the vicinity of the optimal solution after several iterations, which can be proved from Fig. 6 and Fig. 7.

The similarity among LHS, DUD, SDI and the proposed DLU is that they all divide the search space into grids and select some points. It is worth mentioning that there are many differences among these four methods. LHS can prevent the solution from being too aggregated, but it will suffer the curse of dimensionality. DUD randomly selects an initial individual in each diagonal subspace. However, the selected point may appear anywhere in a subspace. Therefore, the choice of the initial population is uncertain. In fact, this does not guarantee the performance of each experimental result. Subsequent experiments also proved that LHS and DUD were not as effective as expected.

The initial solutions generated by SDI and DLU are deterministic. But SDI is restricted by the sequence selection mechanism: when the value of q is fixed, the population size will be completely determined and cannot be freely selected. Most importantly, the validity of the results requires that the value of q should not be too small, which would lead to a large population. However, expensive computation has been a problem that intelligent algorithms struggle to overcome.

Our proposed DLU requires fewer initial individuals to provide extensive information for each dimension. The initial population generated by DLU can improve the robustness of the result (the randomness of metaheuristic algorithms comes from the initial population and iterative process). Another advantage of DLU is that it has good performance in high-dimensional space.

IV. NUMERICAL EXPERIMENTS

The purpose of this paper is to compare the performance of different initialization technologies. Therefore, four classic algorithms are selected for subsequent experiments. In fact, since there are nearly a hundred kinds of metaheuristic optimizers, it is impossible to study all the algorithms. Considering the different search characteristics, differential evolution (DE), particle swarm optimization (PSO), cuckoo search (CS) [54] and artificial bee colony (ABC) [55] are chosen as examples. DE is a representative of evolutionary algorithms, and PSO is a typical representative of swarm intelligence. In addition, CS separates global search from local search and uses Lévy flights instead of the random walk. ABC is another widely used algorithm that also focuses on the ability of local optimization. These four methods have received widespread attentions [56]–[59] and have been successfully applied in engineering and industrial fields.

A. COMPARISON OF DIFFERENT INITIALIZATION METHODS

To compare the effectiveness of the proposed initialization technology, a series of experiments were carried out using nine benchmark functions, as shown in Table 1. These functions are representative and have different properties (such as modal shape). More specifically, f_1, f_2, f_3, and f_4 are continuous unimodal functions, while others are multi-modal functions. These functions are usually used to test the performance of algorithms [60]. The optimal solution of function f_4 is -1, and the optimal solutions of other functions are all 0.

TABLE 1. Benchmark Functions.

Name	Function	Search Range
Rosenbrock	$f_1(X) = \sum_{i=1}^{D-1} [100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2]$	$[-5, 5]^D$
Sphere	$f_2(X) = \sum_{i=1}^{D} x_i^2$	$[-5, 5]^D$
Zakharov	$f_3(X) = \sum_{i=1}^{D} x_i^2 + (\frac{1}{2}\sum_{i=1}^{D} ix_i)^2 + (\frac{1}{2}\sum_{i=1}^{D} ix_i)^4$	$[-100, 100]^D$
Easom	$f_4(X) = [-\prod_{i=1}^{D} \cos(x_i)] * \exp(-\sum_{i=1}^{D} (x_i - \pi)^2)$	$[-100, 100]^D$
Ackley	$f_5(X) = -20\exp(-0.2\sqrt{\frac{1}{D}\sum_{i=1}^{n} x_i^2}) - \exp(\frac{1}{D}\sum_{i=1}^{D} \cos(2\pi x_i)) + 20 + e$	$[-10, 10]^D$
Rastrigin	$f_6(X) = \sum_{i=1}^{D} [x_i^2 - 10\cos(2\pi x_i)] + 10 * D$	$[-5.12, 5.12]^D$
Griewank	$f_7(X) = \frac{1}{4000}\sum_{i=1}^{D} x_i^2 - \prod_{i=1}^{D} \cos(\frac{x_i}{\sqrt{i}}) + 1$	$[-600, 600]^D$
Alpine	$f_8(X) = \sum_{i=1}^{D} \lvert x_i \sin(x_i) + 0.1x_i \rvert$	$[-10, 10]^D$
Schwefel	$f_9(X) = 418.98288727243369 * D - \sum_{i=1}^{D} x_i \sin(\sqrt{\lvert x_i \rvert})$	$[-500, 500]^D$

For the differential evolution (DE) algorithm, there are many different variants. After some preliminary experimental studies, in this paper, we will use a simple algorithm with better performance than the original DE, called parameter adaptive DE (DE-a) [49]. The parameters CR and F are chosen in the set [0.5, 0.6, 0.7, 0.8, 0.9] and [0.4, 0.5, 0.6, 0.7, 0.8], respectively. Firstly, CR and F are randomly initialized in the set of discrete values. If the next generation finds a better solution, previous values of CR and F will continue to be used. Otherwise, the new parameter value will replace the original one. In this way, parameters realize adaptation.

There are two other parameters: population size and the number of iterations. The fact is that DE depends on the number of iterations, a small size of the population can find a better solution after several iterations [49]. Due to the limitations of SDI: the population size cannot be freely chosen. Take a 10-dimensional problem as an example. Even if each dimension is divided into three parts, the total population will exceed 100. This is one of the major drawbacks of SDI. Hence, the comparison of initialization methods: random, uniform, LHS, DLU, OLM and Chaotic (Gauss map), is given first. The convergence curves (except SDI) of 10-dimensional benchmark functions are shown in the following Fig. 8. The termination condition: maximum number of fitness evaluations (FEs) is set to 50,000 and the population size is set to 100.

The following experiments quantitatively compare the performance of these initialization methods, especially include SDI. For all the test functions, three different dimensional scenarios ($D = 10$, $D = 30$ and $D = 100$) are tested. Each experiment runs 20 times independently. We have summarized the results as the 'Best', 'Mean', 'Var' (variance) and 'Dist'. Here, 'Dist' corresponds to the mean distance from the obtained solution x^{find} to the true global optimal solution x^{opt} [49]. That is

$$Dist = \frac{\sum_{j=1}^{TN}\sum_{i=1}^{D}\left|x_{j,i}^{find} - x_i^{opt}\right|}{TN}, \tag{11}$$

where $TN = 20$ denotes the total number of runs for each experiment. This distance metric not only measures the degree of deviation of the results, but also reflects the stability of the obtained solutions.

For fair comparisons, all initialization methods are run under the same experimental environment and parameter settings (except for EFs). Due to the particularity of SDI, we try to minimize the difference in the number of maximum fitness evaluations in the experiments (except for the SDI, the EFs of other methods are exactly the same. To illustrate the advantages of our method, the number of EFs of SDI is slightly higher than that of all other methods). For 10D problems N is set to 1,111 ($q = 11$ for SDI). After an iteration, the experimental results are shown in Table 2.

As seen, SDI and DLU are much better than the other seven approaches for these nine functions. For f_1, f_2, f_3, f_6, f_7, and f_8, the values of 'Best', 'Mean', 'Var', and 'Dist' of SDI and DLU are 0s, which means that the best solution is found in each experiment. Although the 'Best' solutions of other approaches are optimal, the performance of 'Mean', 'Var' and 'Dist' are various. This shows that SDI and DLU are better than others. They can not only improve the search capability of the algorithm, but also make it more robust. For function f_5, SDI and DLU have the same performance. For f_4 and f_9, the 'Best' and 'Mean' indicate that the solutions obtained by DLU are closer to the real optimal solutions than other initialization strategies. For f_9, the values of 'Var' and 'Dist' in the last column (DLU) are the smallest. Hence, DLU is better than SDI.

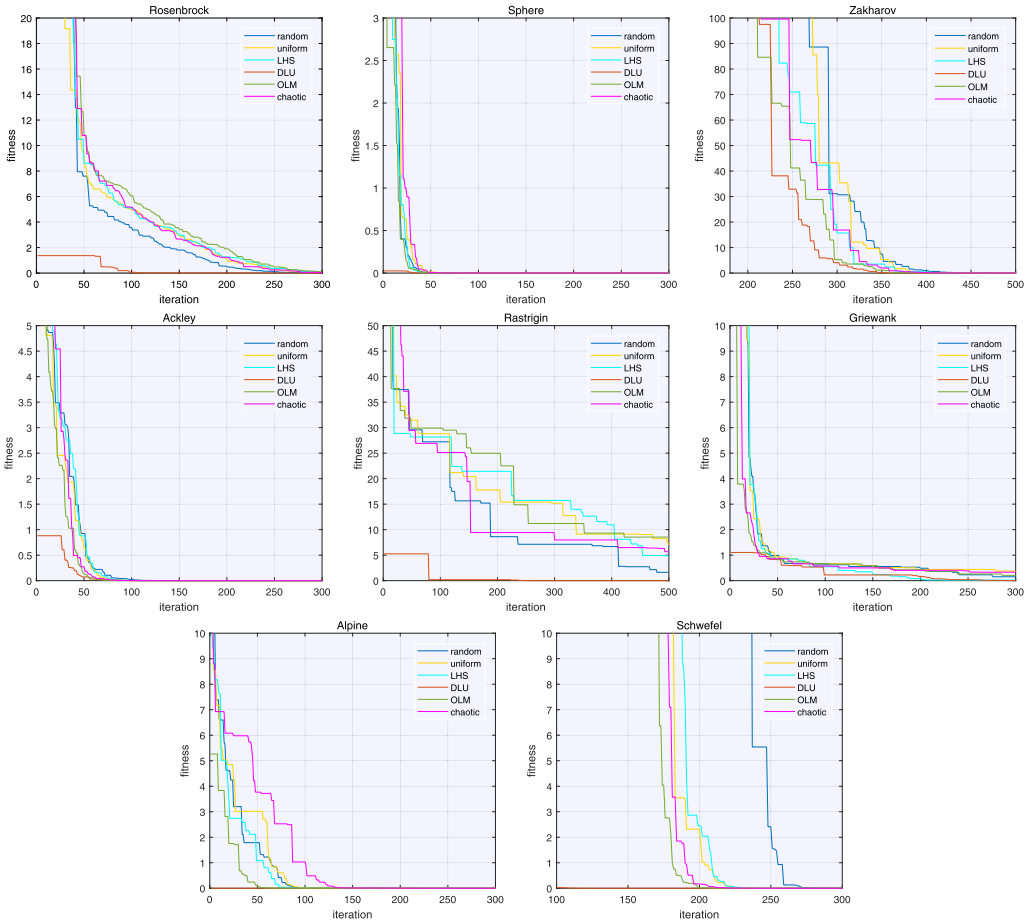


FIGURE 8. The convergence curves of different initialization methods.

For 30-dimensional problems, parameter q of SDI is still set to 11, which means that N is at least 3,311. Hence, for SDI, the maximum number of fitness evaluations (FEs) is set to 9,933. To show the performance of our proposed DLU more clearly. The maximum number of fitness evaluations (FEs) for all other initialization strategies is set to 9,900 and N is set to 100.

The experimental results are shown in Table 3. According to the obtained results, except for function f_5, the 'Best', 'Mean', 'Var' and 'Dist' of DLU performs better than other initialization methods. Our proposed DLU not only finds the optimal solution accurately, but also improves the robustness of the algorithm. Although it has only 100 initial individuals, its performance is superior to others. In fact, an important advantage of DLU is that a small number of initial individuals carry a large amount of information in the search space and find the optimal solution faster. It should be noted that some commonly used initialization methods, such as Random, Latin hypercube sampling and Uniform, etc., are less effective and seem to be inconsistent with the usual results. The reason is that for a fair comparison, the number of iterations is relatively small here.

For 100-dimensional problems, when $q = 11$, the population number of SDI is 11,011. To make a fair comparison, the maximum number of fitness evaluations (FEs) of other approaches is set to 11,000. In addition to SDI, the population sizes and the maximum number of iterations are set to 100 and 110. The results are shown in Table 4.

It can be seen from the experimental results that the performance of DLU is still better than other initialization strategies, and it has not been greatly affected after the problem dimension is increased. On the contrary, the performance of other strategies will decrease after the dimension is increased. Another important advantage of DLU is that its performance does not decrease significantly with increasing dimensions.

B. TESTING PROPOSED METHOD WITH OTHER METAHEURISTIC OPTIMIZER

In this subsection, we will test the effectiveness of the proposed DLU for another three algorithms: an improved particle swarm optimization (PSO) with inertia weight (PSO-w) [61], cuckoo search (CS) and artificial bee colony (ABC). As three important representatives of the swarm-based algorithm, they have different search characteristics. The p_a in CS is a switching probability, which is used to balance global search and local search. *limit* is a control parameter in ABC, that is, when the *limit* is exceeded, the food source will no longer

TABLE 2. Comparisons of different initialization methods (10D).

Fun	Value	Random	Uniform	LHS	DUD	OLM	Chaotic1	Chaotic2	SDI	DLU
f_1	Best	0	0	0	0	0	0	0	0	0
	Mean	1.65E+03	1.81E+03	1.66E+03	1.63E+03	1.55E+03	1.28E+03	1.60E+03	0	0
	Var	4.59E+06	5.93E+06	4.52E+06	3.94E+06	3.27E+06	2.16E+06	3.38E+06	0	0
	Dist	1.10E+01	1.17E+01	1.19E+01	1.17E+01	1.10E+01	1.53E+01	1.10E+01	0	0
f_2	Best	0	0	0	0	0	0	0	0	0
	Mean	6.75	6.31	6.22	7.29	6.09	4.24	6.96	0	0
	Var	5.17E+01	4.94E+01	4.64E+01	5.94E+01	4.46E+01	1.97E+01	5.71E+01	0	0
	Dist	9.43	8.92	9.03	9.49	8.89	8.13	9.87	0	0
f_3	Best	0	0	0	0	0	0	0	0	0
	Mean	6.20E+03	7.15E+03	6.96E+03	5.68E+03	6.06E+03	5.96E+03	7.88E+03	0	0
	Var	4.99E+07	5.87E+07	5.98E+07	3.65E+07	4.39E+07	4.36E+07	6.94E+07	0	0
	Dist	2.71E+02	3.05E+02	2.86E+02	2.36E+02	2.75E+02	2.77E+02	3.19E+02	0	0
f_4	Best	0	0	0	0	0	0	0	-1.37E-43	-0.97
	Mean	0	0	0	0	0	0	0	-6.85E-44	-0.46
	Var	0	0	0	0	0	0	0	4.82E-87	0.21
	Dist	5.08E+02	5.08E+02	5.03E+02	4.95E+02	4.84E+02	5.11E+09	4.84E+02	3.14E+01	0.77
f_5	Best	0	0	0	0	0	0	0	-8.88E-16	-8.88E-16
	Mean	4.31	4.54	4.41	4.39	4.34	3.90	4.26	-4.44E-16	-4.44E-16
	Var	1.97E+01	2.15E+01	2.02E+01	2.02E+01	1.96E+01	1.57E+01	1.92E+01	2.02E-31	2.02E-31
	Dist	1.90E+01	1.99E+01	1.78E+01	1.83E+01	1.81E+01	1.67E+01	1.75E+01	0	0
f_6	best	0	0	0	0	0	0	0	0	0
	Mean	3.70E+01	3.82E+01	3.58E+01	3.40E+01	3.50E+01	3.09E+01	3.95E+01	0	0
	Var	1.45E+03	1.52E+03	1.36E+03	1.19E+03	1.30E+03	1.02E+03	1.63E+03	0	0
	Dist	1.40E+01	1.39E+01	1.40E+01	1.64E+01	1.35E+01	1.13E+01	1.55E+01	0	0
f_7	Best	0	0	0	0	0	0	0	0	0
	Mean	2.51E+01	2.48E+01	2.28E+01	2.60E+01	2.19E+01	1.63E+01	2.36E+01	0	0
	Var	7.55E+02	7.43E+02	6.16E+02	7.73E+02	5.53E+02	2.94E+02	6.67E+02	0	0
	Dist	1.10E+03	1.13E+03	1.10E+03	1.16E+03	1.07E+03	9.99E+02	1.06E+03	0	0
f_8	Best	0	0	0	0	0	0	0	0	0
	Mean	4.03	4.21	3.87	3.99	3.86	3.50	3.95	0	0
	Var	1.74E+01	1.87E+01	1.60E+01	1.71E+01	1.56E+01	1.33E+01	1.65E+01	0	0
	Dist	2.98E+01	2.86E+01	3.12E+01	2.95E+01	3.12E+01	2.67E+01	2.87E+01	0	0
f_9	Best	0	0	0	0	0	-4.07E+126	0	0	0
	Mean	1.09E+03	1.05E+03	1.04E+03	1.10E+03	1.02E+03	-1.02E+125	1.14E+03	2.69E+02	0.39E-02
	Var	1.23E+06	1.16E+06	1.12E+06	1.26E+06	1.09E+06	4.15E+251	1.35E+06	7.42E+04	1.50E-03
	Dist	2.86E+03	2.90E+03	2.92E+03	3.38E+03	2.89E+03	2.38E+125	3.50E+03	2.10E+02	2.48

be exploited and is assumed to be abandoned. The parameter settings of three algorithms are shown in Table 5.

The experiments use 30D benchmark functions as the test objects. Each experiment runs for 20 independent times. The termination conditions (the maximum number of fitness evaluations (FEs)) for all experiments should be set approximately the same. For SDI, the population size is 3,311(when $q = 11$), and the number of iterations is set to 20, then the maximum number of FEs is 66,220. In contrast, the maximum number of FEs for other initialization strategies is set to 66,000. Specifically, for PSO and CS, the population size is set to 500 and the number of iterations is 132. For ABC, the population size and the number of iterations are set to 100 and 660, respectively. The test results are shown in Table 6, Table 7 and Table 8.

The experimental results are similar to those of DE algorithm, which proves the superiority of the proposed DLU. More specifically, for function f_1, f_2, f_3, f_6, f_7 and f_8, the initialization methods SDI and DLU not only find the optimal solution, but also succeed every time (the values of 'Mean' and 'Var' are 0s). That means, in the same case, SDI and DLU are obviously more effective than other approaches. For function f_4, the optimal solution is -1. In fact, except for SDI and DLU, other approaches fall into the local optimum. The 'Best', 'Mean' value of DLU is closer to the true optimum than SDI. The 'Dist' value of DLU is smaller than that of others, including SDI. These results show that the solutions found by DLU method have higher accuracy. For PSO and CS, although most of the test initialization strategies have found the optimal solution of function f_9, the robustness of these methods (except for DLU) is not good, so the values of mean and variance are enormous. For ABC, DLU performs best, as can be seen from the smaller 'Best', 'Mean' and 'Var' of function f_9. The performance of DLU is the best of all test approaches.

In summary, under the same conditions (the maximum number of FEs is 66,000), initialization methods: Random, Uniform, LHS, DUD, Chaotic1, and Chaotic2 perform worse than our proposed DLU. Even when the maximum FEs of SDI equals 63,220 (there are 220 FEs more than other methods), the initialization effect is still slightly lower than that of the DLU. Hence, the proposed method is the most effective.

Furthermore, based on the obtained results of 30-dimensional benchmark functions, the Friedman rank test [62] is used to calculate the rank value and test whether there are significant differences in various initialization methods by p-value. The null hypothesis is that all the initialization strategies are equally effective. Table 9 presents details of the rankings about DE-a, PSO, CS and ABC. The p values are far less than 0.05. The results clearly show that these

TABLE 3. Comparisons of different initialization methods (30D).

Fun	Value	Random	Uniform	LHS	DUD	OLM	Chaotic1	Chaotic2	SDI	DLU
f_1	Best	0	0	0	0	0	0	0	0	0
	Mean	5.17E+01	5.04E+01	4.32E+01	4.32E+01	4.76E+01	4.15E+01	3.83E+01	0	0
	Var	3.67E+03	3.30E+03	2.38E+03	2.44E+03	3.32E+03	2.58E+03	1.87E+03	0	0
	Dist	2.52E+01	2.62E+01	2.74E+01	2.68E+01	2.68E+01	2.73E+01	2.76E+01	0	0
f_2	Best	0	0	0	0	0	0	0	0	0
	Mean	4.92E-02	4.48E-01	5.31E-02	4.54E-02	5.42E-02	6.38E-02	3.90E-02	0	0
	Var	4.41E-03	3.55E-03	4.28E-03	5.80E-03	7.87E-03	8.63E-03	3.12E-03	0	0
	Dist	1.33	1.27	1.38	1.24	1.33	1.47	1.17	0	0
f_3	Best	0	0	0	0	0	0	0	0	0
	Mean	2.25E+04	2.28E+04	2.18E+04	2.16E+04	2.25E+04	2.03E+04	2.43E+04	0	0
	Var	5.42E+08	5.65E+08	5.14E+08	4.99E+08	5.47E+08	4.54E+08	6.26E+08	0	0
	Dist	9.22E+02	9.37E+02	9.07E+02	9.09E+02	9.16E+02	8.89E+02	9.78E+02	0	0
f_4	Best	0	0	0	0	0	0	0	-2.57E-129	-7.24E-16
	Mean	0	0	0	0	0	0	0	-1.29E-129	-3.62E-16
	Var	0	0	0	0	0	0	0	1.70E-258	1.34E-31
	Dist	1.30E+03	1.38E+03	1.34E+03	1.34E+03	1.32E+03	1.31E+03	1.33E+03	9.42E+01	2.58+01
f_5	Best	0	0	0	0	0	0	0	-8.88E-16	-8.88E-16
	Mean	6.10E-01	5.22E-01	6.28E-01	6.67E-01	7.20E-01	5.52E-01	7.10E-01	-4.44E-16	-4.44E-16
	Var	5.24E-01	3.74E-01	5.03E-01	5.79E-01	6.99E-01	3.97E-01	6.41E-01	2.02E-31	2.02E-31
	Dist	3.20	2.81	3.26	3.53	3.81	2.90	3.70	0	0
f_6	best	0	0	0	0	0	0	0	0	0
	Mean	9.90E+02	1.02E+02	1.02E+02	1.02E+02	1.01E+02	6.9E+01	1.03E+02	0	0
	Var	1.02E+04	1.08E+04	1.08E+04	1.07E+04	1.06E+04	9.72E+03	1.11E+04	0	0
	Dist	3.03E+01	3.18E+01	3.35E+01	3.16E+01	3.24E+01	3.20E+01	3.32E+01	0	0
f_7	Best	0	0	0	0	0	0	0	0	0
	Mean	6.66E-01	6.43E-01	7.01E-01	6.84E-01	6.67E-01	6.56E-01	6.83E-01	0	0
	Var	4.71E-01	4.41E-01	5.47E-01	5.01E-01	4.79E-01	4.56E-01	5.27E-01	0	0
	Dist	1.58E+02	1.39E+02	1.66E+02	1.60E+02	1.54E+02	1.50E+02	1.58E+02	0	0
f_8	Best	0	0	0	0	0	0	0	0	0
	Mean	6.40	6.10	6.68	5.55	6.41	6.24	6.47	0	0
	Var	4.72E+01	4.52E+01	5.42E+01	3.69E+01	4.63E+01	4.44E+01	4.69E+01	0	0
	Dist	3.06E+01	3.18E+01	2.78E+01	2.88E+01	3.08E+01	3.03E+01	3.12E+01	0	0
f_9	Best	0	0	0	0	0	-1.78E+175	0	0	0
	Mean	3.83E+03	3.74E+03	3.84E+03	3.85E+03	3.80E+03	-3.52E+38	3.84E+03	7.11E+02	1.78
	Var	1.51E+07	1.45E+07	1.51E+07	1.52E+07	1.49E+07	4.83E+78	1.52E+07	5.58E+05	3.23
	Dist	9.94E+03	1.03E+04	1.00E+04	1.06E+04	1.01E+04	7.74E+38	1.05E+04	1.59E+03	2.91E+01

nine initialization strategies have significant differences at $\alpha = 0.05$ level. Although these four metaheuristic algorithms represent different types and have different iteration mechanisms, all the algorithms were improved by using the proposed initialization technology. In addition, the results prove the superiority of the proposed DLU.

C. SOLVING MULTI-OBJECTIVE PROBLEMS

Metaheuristic optimizers can solve multi-objective problems(MOPs) by their powerful search capability and population-based framework [63]. Although the existing multi-objective optimization algorithm obtains successful results in part of the problems, the practical performance still needs to be improved. In fact, in multi-objective problems, the initial population has a huge impact on the performance of algorithms, and many technical details in this topic still need to be solved.

The widely used MOP framework is the multi-objective evolutionary algorithm based on decomposition (MOEA/D) [42], which decomposes MOP into multiple single-object sub-problems and optimizes the solutions of different sub-problems in a collaborative manner. MOEA/D algorithm has been applied to many multi-objective problems [64]–[67].

Based on the previous experiments, it can be seen that SDI is a competitive method (SDI has the best performance except for our proposed DLU). In addition, Uniform initialization is the most widely used strategy in multi-objective optimization. Therefore, in this subsection, we will compare the impact of Uniform, SDI, and DLU under the MOEA/D framework on solving bi-objective benchmark problems. Some well-known benchmark functions (ZDT1, ZDT2, ZDT3, ZDT4, and ZDT6) were tested. To provide fair comparisons, the maximum number of FEs is set to 10,000. The population size N is set to 100 and the maximum number of iterations is set to 100. The parameter settings of MOEA/D are the same as those used in the proposed paper [42].

Because of the particularity of SDI method, we carried out two types of SDI experiments: take 30 dimensional as an example, one group is called SDI, in this group the population size is still 3,311, the maximum number of iterations is 3. The other group is called SDI-opt, which first produces 3,311 individuals. According to the non-dominant sorting, 100 individuals are selected as the initial population. Thus, the number of remaining iterations is set to 66 (the maximum number of FEs is still set to 9,911).

For multi-objective problems, a set of Pareto optimal solutions is obtained by balancing different objectives. We hope that this set of solutions can be close to the real optimal solution set (the deviation from the real optimal solution set is the smallest), and the diversity of the solutions is good (evenly and dispersedly distributed in the Pareto Front). Therefore,

TABLE 4. Comparisons of different initialization methods (100D).

Fun	Value	Random	Uniform	LHS	DUD	OLM	Chaotic1	Chaotic2	SDI	DLU
f_1	Best	0	0	0	0	0	0	0	0	0
	Mean	3.01E+03	3.18E+03	3.13E+03	3.25E+03	3.14E+03	3.64E+03	3.18E+03	0	0
	Var	1.10E+07	1.22E+07	1.14E+07	1.38E+07	1.09E+07	1.66E+07	1.24E+07	0	0
	Dist	8.13E+01	8.22E+01	8.35E+01	8.34E+01	8.47E+01	9.09E+01	8.46E+01	0	0
f_2	Best	0	0	0	0	0	0	0	0	0
	Mean	9.44	1.02E+01	1.06E+01	8.86	9.52	1.18E+01	1.06E+01	0	0
	Var	9.90E+01	1.33E+02	1.31E+02	9.35E+01	1.09E+02	1.67E+01	1.30E+01	0	0
	Dist	3.43E+01	3.54E+01	3.66E+01	3.34E+01	3.42E+01	3.82E+01	3.64E+01	0	0
f_3	Best	0	0	0	0	0	0	0	0	0
	Mean	1.07E+05	1.12E+05	1.14E+05	1.14E+05	1.12E+05	1.08E+05	1.04E+05	0	0
	Var	1.19E+10	1.30E+10	1.36E+10	1.39E+10	1.31E+10	1.21E+10	1.15E+10	0	0
	Dist	3.81E+03	3.89E+03	3.99E+03	3.87E+03	3.88E+03	3.79E+03	3.75E+03	0	0
f_4	Best	0	0	0	0	0	0	0	-8.80E-301	-4.68E-23
	Mean	0	0	0	0	0	0	0	-2.20E-302	-1.17E-24
	Var	0	0	0	0	0	0	0	0	5.47E-47
	Dist	4.42E+03	4.59E+03	4.47E+03	4.43E+03	4.42E+03	4.54E+03	4.51E+03	9.81E+03	8.36E+01
f_5	Best	0	0	0	0	0	0	0	-8.88E-16	-8.88E-16
	Mean	2.47	2.55	2.58	2.56	2.57	2.66	2.57	-4.44E-16	-4.44E-16
	Var	6.39	6.75	6.92	6.80	6.79	7.33	6.87	2.02E-31	2.02E-31
	Dist	7.17E+01	7.60E+01	7.82E+01	7.61E+01	7.65E+01	8.13E+01	7.80E+01	0	0
f_6	best	0	0	0	0	0	0	0	0	0
	Mean	4.85E+02	4.78E+02	4.86E+02	4.92E+02	4.87E+02	4.89E+02	4.93E+02	0	0
	Var	2.42E+05	2.35E+05	2.42E+05	2.49E+05	2.44E+05	2.45E+05	2.50E+05	0	0
	Dist	1.22E+02	1.21E+02	1.27E+02	1.25E+02	1.22E+02	1.26E+02	1.22E+02	0	0
f_7	Best	0	0	0	0	0	0	0	0	0
	Mean	3.81E+01	3.55E+01	3.66E+01	3.55E+01	3.65E+01	4.73E+01	3.49E+01	0	0
	Var	1.69E+03	1.52E+03	1.49E+03	1.41E+03	1.54E+03	2.70E+03	1.41E+03	0	0
	Dist	4.32E+03	4.16E+03	4.27E+03	4.13E+03	4.21E+03	4.83E+03	4.13E+03	0	0
f_8	Best	0	0	0	0	0	0	0	0	0
	Mean	4.47E+01	4.32E+01	4.18E+01	4.28E+01	4.20E+01	4.28E+01	4.17E+01	0	0
	Var	2.09E+03	1.95E+03	1.82E+03	1.92E+03	1.84E+03	1.90E+03	1.82E+03	0	0
	Dist	1.66E+02	1.57E+02	1.56E+02	1.65E+02	1.58E+03	1.63E+02	1.58E+02	0	0
f_9	Best	0	0	0	0	0	-1.78E+175	0	0	0
	Mean	1.66E+04	1.67E+04	1.66E+04	1.65E+04	1.65E+04	-2.55E+138	1.66E+04	2.09E+02	5.92
	Var	2.84E+08	2.85E+08	2.82E+08	2.80E+08	2.81E+08	2.60E+278	2.81E+08	5.44E+06	3.59E+01
	Dist	3.79E+04	3.84E+04	3.76E+04	3.89E+04	3.84E+04	1.14E+139	3.96E+04	6.84E+03	9.69E+01

two performance indicators are used to evaluate the above two objectives, respectively.

The first metric γ measures the consistency between the obtained solution set and the true Pareto optimal solutions. Let P be a set of uniformly distributed points along the true Pareto Front. Let C be the set of obtained solutions. The γ is defined as

$$\gamma = \sum_{i=1}^{N} \min(d(\vec{x}_i, v)),\ v \in P,\ x_i \in C.$$

where $d(\vec{x}_i, v)$ means the distances between x_i and v. The higher the value of γ, the greater the deviation of the found solution set from the true Pareto Front. The number of set P is recommended to be set to 500 [68].

The second metric σ is inspired by the metric Δ in paper [68], but different from that mentioned. Because d_s and d_e reflect whether the two ends of the obtained front are in the right positions. If not, the uniform distribution is meaningless. So the weight of this item should be larger. The σ is defined as:

$$\sigma = d_s + d_e + \frac{\sum_{i=1}^{N-1} \left| d_i - \bar{d} \right|}{(Q-1)\bar{d}}$$

TABLE 5. Parameter settings for PSO-w and CS.

PSO-w	$c_1 = 1.5\,, c_2 = 1.5\,, w = 0.8$
CS	$p_a = 0.25$
ABC	$limit = 3000$

where d_s and d_e are the Euclidean distances between the extreme solutions of the true PF and the boundary solutions of the obtained non-dominated set, d_i denotes the distance between point i and point $i+1$, where $i = 1, 2, \cdots, N-1$. The parameter Q denotes the number of different solutions in the obtained non-dominated set. This index reflects the uniformity of the solution distribution. The more uniform the distribution, the better the diversity of solutions.

Each experiment runs 20 times, and the experimental results are summarized in Table 10. It also indicates that DLU performs well in solving multi-objective problems. For problems ZDT2 and ZDT6, the values of both the metric γ and σ show that DLU is better than other methods. Especially, for ZDT2, the SDI often loses its effectiveness, while DLU can find the smooth and uniform Pareto Front. For ZDT3 and ZDT4, the values of γ indicate that the non-dominated solutions obtained by DLU are closer to the true Pareto Front than other approaches. But the diversity of DLU is slightly worse than Uniform (for ZDT3) and SDI (for ZDT4).

TABLE 6. Comparisons of different initialization methods with PSO-w (30D).

Fun	Value	Random	Uniform	LHS	DUD	OLM	Chaotic1	Chaotic2	SDI	DLU
f_1	Best	0	0	0	0	0	0	0	0	0
	Mean	1.33E+01	1.30E+01	1.29E+01	1.32E+01	1.29E+01	1.43E+01	1.30E+01	0	0
	Var	1.82E+02	1.73E+02	1.73E+02	1.78E+02	1.72E+02	2.71E+02	1.75E+02	0	0
	Dist	2.78E+01	2.73E+01	2.71E+01	2.76E+01	2.71E+01	2.64E+01	2.73E+01	0	0
f_2	Best	0	0	0	0	0	0	0	0	0
	Mean	4.60E-07	4.26E-07	4.22E-07	4.36E-07	5.21E-07	6.15E-07	4.18E-07	0	0
	Var	3.20E-13	3.08E-13	3.33E-13	2.60E-13	4.33E-13	8.21E-13	3.33E-13	0	0
	Dist	4.09E-03	3.85E-03	3.89E-03	4.00E-03	4.31E-03	4.57E-03	3.81E-03	0	0
f_3	Best	0	0	0	0	0	0	0	0	0
	Mean	4.58E+02	4.72E+02	4.64E+02	4.05E+02	3.31E+02	5.33E+02	5.34E+02	0	0
	Var	2.78E+05	2.99E+05	4.14E+05	2.29E+05	1.68E+05	4.06E+05	4.76E+05	0	0
	Dist	1.27E+02	1.31E+02	1.23E+02	1.18E+02	1.09E+02	1.37E+02	1.36E+02	0	0
f_4	Best	0	0	0	0	0	0	0	-1.29E-03	-9.26E-01
	Mean	0	0	0	0	0	0	0	-3.94E-05	-1.61E-01
	Var	0	0	0	0	0	0	0	4.32E-08	6.70E-02
	Dist	1.53E+03	1.51E+03	1.52E+03	1.41E+03	1.52E+03	2.64E+50	1.58E+03	4.42	2.10E+01
f_5	Best	0	0	0	0	0	0	0	-8.88E-16	-8.88E-16
	Mean	6.94E-01	7.27E-01	7.65E-01	7.12E-01	4.84E-01	7.10E-01	7.01E-01	-4.44E-16	-4.44E-16
	Var	7.09E-01	8.15E-01	8.09E-01	6.97E-01	6.12E-01	7.66E-01	6.86E-01	2.02E-31	2.02E-31
	Dist	4.88	5.64	5.76	4.97	3.54	5.14	4.84	0	0
f_6	best	0	0	0	0	0	0	0	0	0
	Mean	1.70E+01	1.62E+01	1.57E+01	1.57E+01	1.55E+01	1.75E+01	1.90E+01	0	0
	Var	3.50E+02	3.41E+02	2.86E+02	2.88E+02	2.83E+02	3.53E+02	3.95E+02	0	0
	Dist	2.28E+01	2.21E+01	2.20E+02	2.15E+02	2.20E+02	2.36E+01	2.43E+02	0	0
f_7	Best	0	0	0	0	0	0	0	0	0
	Mean	7.13E-03	5.62E-03	7.18E-03	5.01E-03	4.83E-03	9.82E-03	9.19E-03	0	0
	Var	1.40E-04	7.51E-05	1.18E-04	9.89E-05	4.11E-05	2.05E-04	2.17E-04	0	0
	Dist	9.77	6.97	9.23	5.93	7.28	1.16E+01	1.05E+01	0	0
f_8	Best	0	0	0	0	0	0	0	0	0
	Mean	1.25E-01	2.05E-01	1.68E-01	1.23E-01	9.87E-02	2.15E-01	1.70E-01	0	0
	Var	3.28E-02	1.19E-01	5.18E-02	3.00E-02	1.96E-02	1.73E-01	6.08E-02	0	0
	Dist	2.16E+01	2.90E+01	2.55E+01	2.22E+01	2.01E+01	2.65E+01	2.49E+01	0	0
f_9	Best	0	0	0	0	0	-1.73E+191	0	0	0
	Mean	1.88E+03	1.97E+03	1.93E+03	2.02E+03	1.83E+03	-4.33E+189	2.02E+03	3.63E+03	2.14E-02
	Var	3.76E+06	4.13E+06	3.91E+06	4.27E+06	3.54E+06	Inf	4.31E+06	3.70E+03	8.15E-04
	Dist	9.69E+03	9.97E+03	1.01E+03	1.02E+04	9.60E+04	8.91E+189	1.13E+04	2.85E+02	2.37

Four indicators ('Mean', 'Var', 'γ' and 'σ') analyze the simulation results from different perspectives. Hence, the Friedman tests were used to validate the results. The null hypothesis is that these four initialization technologies are equally effective.

The Friedman ranks and p values are shown in Table 11. From these results, the p values of ZDT1, ZDT2, ZDT3 and ZDT4 are less than the given significance level of 0.05, the null hypothesis is rejected and four approaches are considered to be significantly different. It is clear that most of the benchmarks are considered improved by using the proposed initialization strategy but, for ZDT6, is not significantly affected by using different initialization approaches. One reason for this might be that the MOEA/D framework itself has particularly good performance. Moreover, the space formed by the Pareto set is quite small (the search space of these benchmark functions is not large). Therefore, effective improvement is very limited.

Take these five functions as observation samples, the p value is 0.0004, which is far less than 0.05. So it can be considered that these four initialization approaches will have a significant impact on the performance of the algorithm. According to the Friedman rank, it can conclude that DLU is better than the other three methods. Moreover, the Random and Latin hypercube sampling have similar effects. SDI method has improved compared with the former two, but our method is the best. It is worth emphasizing that it is unbelievably valuable to improve the performance of the algorithm through a simple initial step without increasing the computational cost.

Furthermore, to show the difference among different initialization methods more intuitively, the distribution of the final fitness values of the non-dominated solutions for each function is shown in Fig. 9 to Fig. 13. As indicated by the results, the Pareto Front of DLU is continuous and uniform. Under the same conditions, the performance of SDI method is not good due to the influence of iteration times. For ZDT1, ZDT3, ZDT4, the effect of initialization SDI-opt is similar to that of the Random method. DLU is usually better than random initialization. In summary, it was obvious that MOEA/D with the proposed DLU was superior to that with the Uniform, SDI and SDI-opt initialization.

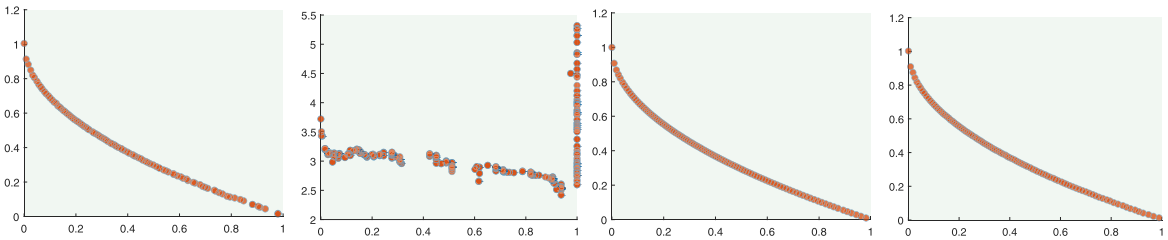
V. DISCUSSIONS

A. ADDITIONAL EXPERIMENTAL RESULTS

In this section, some of the latest benchmark functions from CEC2014 and CEC2017 are also used to test the performance of DLU. Due to the addition of rotation,

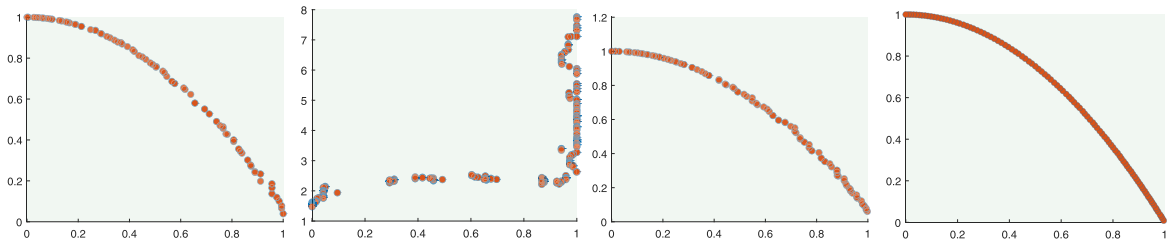
TABLE 7. Comparisons of different initialization methods with CS (30D).

Fun	Value	Random	Uniform	LHS	DUD	OLM	Chaotic1	Chaotic2	SDI	DLU
f_1	Best	0	0	0	0	0	0	0	0	0
	Mean	2.86E+03	2.99E+03	2.99E+03	2.81E+03	2.53E+03	3.04E+03	3.31E+03	0	0
	Var	9.02E+06	1.01E+07	9.99E+06	8.71E+06	7.21E+06	1.03E+07	1.22E+07	0	0
	Dist	3.07E+01	3.12E+01	3.19E+01	3.07E+01	3.01E+01	2.95E+01	3.21E+01	0	0
f_2	Best	0	0	0	0	0	0	0	0	0
	Mean	9.46	8.94	8.84	9.32	8.35	9.74	9.62	0	0
	Var	9.76E+01	8.63E+01	8.37E+01	9.24E+01	7.54E+01	1.03E+02	9.95E+01	0	0
	Dist	1.87E+01	1.86E+01	1.86E+01	1.87E+01	1.77E+01	1.96E+01	1.92E+01	0	0
f_3	Best	0	0	0	0	0	0	0	0	0
	Mean	2.86E+04	2.76E+04	2.87E+04	2.98E+04	2.59E+04	2.59E+04	3.04E+04	0	0
	Var	8.53E+08	8.13E+08	8.61E+08	9.45E+08	7.15E+08	7.23E+08	9.67E+08	0	0
	Dist	1.08E+03	1.03E+03	1.09E+03	1.08E+03	9.98E+02	9.78E+02	1.10E+02	0	0
f_4	Best	0	0	0	0	0	0	0	-2.70E-79	-5.08E-09
	Mean	0	0	0	0	0	0	0	-6.76E-81	-1.28E-10
	Var	0	0	0	0	0	0	0	1.83E-159	6.44E-19
	Dist	1.51E+03	1.51E+03	1.51E+03	1.48E+03	1.50E+03	8.15E+12	1.53E+03	8.98E+02	2.23+02
f_5	Best	0	0	0	0	0	0	0	-8.88E-16	-8.88E-16
	Mean	4.01	4.08	4.03	4.01	4.01	4.04	4.15	-4.44E-16	-4.44E-16
	Var	1.66E+01	1.72E+01	1.68E+01	1.66E+01	1.66E+01	1.68E+01	1.77E+01	2.02E-31	2.02E-31
	Dist	4.75E+01	4.87E+01	4.79E+01	4.87E+01	4.82E+01	4.84E+01	5.01E+01	0	0
f_6	best	0	0	0	0	0	0	0	0	0
	Mean	9.34E+01	9.52E+01	9.46E+01	9.37E+01	9.00E+01	8.89E+01	9.61E+01	0	0
	Var	8.97E+03	9.32E+03	9.20E+03	9.04E+03	8.36E+03	8.15E+03	9.50E+03	0	0
	Dist	4.66E+01	4.74E+01	4.60E+01	4.48E+01	4.41E+01	4.30E+01	4.62E+01	0	0
f_7	Best	0	0	0	0	0	0	0	0	0
	Mean	3.38E+01	3.29E+01	3.44E+01	3.45E+01	3.07E+01	3.22E+01	3.63E+01	0	0
	Var	1.21E+03	1.20E+03	1.26E+01	1.25E+03	1.03E+03	1.14E+03	1.40E+03	0	0
	Dist	2.23E+03	2.23E+03	2.31E+03	2.31E+03	2.11E+03	2.21E+03	2.35E+03	0	0
f_8	Best	0	0	0	0	0	0	0	0	0
	Mean	8.96	8.99	8.87	9.09	8.47	8.70	9.05	0	0
	Var	8.32E+01	8.35E+01	8.11E+01	8.52E+01	7.42E+01	7.80E+01	8.45E+01	0	0
	Dist	1.16E+01	1.03E+01	1.14E+01	1.14E+01	1.02E+01	1.07E+01	1.15E+01	0	0
f_9	Best	0	0	0	0	0	-1.73E+191	0	0	0
	Mean	2.51E+03	2.52E+03	2.52E+03	2.53E+03	2.53E+03	-1.33E+126	2.61E+03	2.80E+02	1.69
	Var	6.47E+06	6.54E+06	6.52E+06	6.59E+06	6.58E+06	7.02E+253	6.98E+06	9.23E+04	3.02
	Dist	1.01E+04	1.08E+04	1.01E+04	9.99E+03	1.03E+04	2.71E+126	1.18E+04	8.41E+02	2.80E+01



(a) Uniform initialization (b) SDI (c) SDI_opt (d) DLU

FIGURE 9. Non-dominated front with MOEA/D on ZDT1.



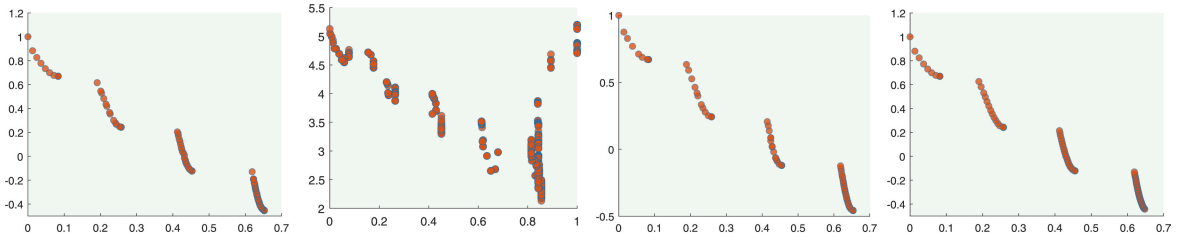
(a) Uniform initialization (b) SDI (c) SDI_opt (d) DLU

FIGURE 10. Non-dominated front with MOEA/D on ZDT2.

combination, and more complex function forms, CEC2014 and CEC2017 benchmark suites have always been more difficult to solve. They all contain 30 test problems, such as unimodal, multi-modal, hybrid, and composition functions. Obviously,

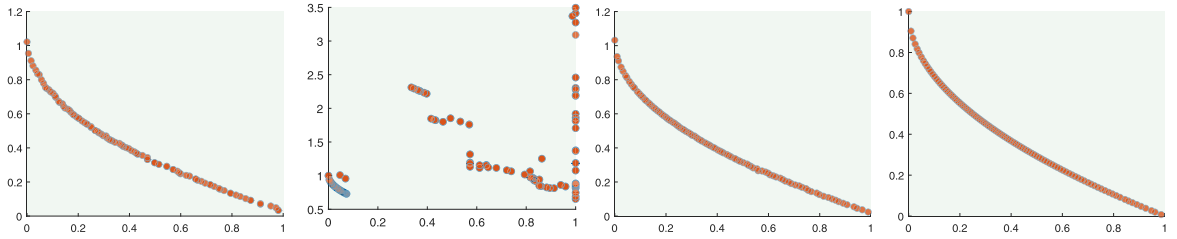
TABLE 8. Comparisons of different initialization methods with ABC (30D).

Fun	Value	Random	Uniform	LHS	DUD	OLM	Chaotic1	Chaotic2	SDI	DLU
f_1	Best	1.37E+05	1.24E+05	1.02E+05	1.53E+05	9.30E+04	1.21E+05	9.06E+04	0	0
	Mean	1.68E+05	1.68E+05	1.56E+05	1.78E+05	1.53E+05	1.95E+07	1.73E+05	0	0
	Var	4.61E+08	4.34E+08	9.16E+08	6.54E+07	7.70E+08	6.23E+14	1.53E+09	0	0
	Dist	5.98E+01	6.12E+01	5.96E+01	6.20E+01	5.86E+01	1.12E+02	6.27E+01	0	0
f_2	Best	1.27E+02	1.20E+02	1.29E+02	1.28E+02	1.18E+02	8.46E+01	1.34E+02	0	0
	Mean	1.49E+02	1.53E+02	1.45E+02	1.40E+02	1.51E+02	8.12E+02	1.59E+02	0	0
	Var	2.21E+02	1.24E+02	1.42E+02	3.25E+01	1.70E+02	2.49E+05	1.56E+02	0	0
	Dist	5.55E+01	5.61E+01	5.40E+01	5.31E+01	5.55E+01	8.96E+01	5.74E+01	0	0
f_3	Best	6.67E+04	7.61E+04	7.61E+04	8.30E+04	6.48E+04	6.46E+09	7.28E+04	0	0
	Mean	2.57E+05	1.75E+05	1.04E+05	1.23E+05	1.93E+05	1.02E+15	2.39E+06	0	0
	Var	4.22E+11	3.08E+10	7.59E+08	6.18E+08	2.19E+11	5.82E+30	7.77E+13	0	0
	Dist	1.39E+03	1.43E+03	1.45E+03	1.64E+03	1.37E+03	2.93E+03	1.45E+03	0	0
f_4	Best	0	0	0	0	0	0	0	-2.57E-129	-1.35E-15
	Mean	0	0	0	0	0	0	0	-2.57E-129	-7.55E-16
	Var	0	0	0	0	0	0	0	4.32E-289	1.97E-32
	Dist	1.47E+03	1.52E+03	1.51E+03	1.38E+03	1.51E+03	1.43E+03	1.56E+03	9.42E+02	2.57E+02
f_5	Best	1.22E+01	1.23E+01	1.27E+01	1.26E+01	1.21E+01	1.29E+01	1.23E+01	-8.88E-16	-8.88E-16
	Mean	1.34E+01	1.34E+01	1.34E+01	1.29E+01	1.33E+01	1.88E+01	1.35E+01	-8.88E-16	-8.88E-16
	Var	1.66E-01	1.47E-01	1.37E-01	1.35E-02	1.62E-01	4.62	2.34E-01	0	0
	Dist	1.09E+02	1.10E+02	1.09E+02	1.09E+02	1.09E+02	2.53E+02	1.12E+02	0	0
f_6	best	3.46E+02	3.66E+02	3.29E+02	3.91E+02	3.52E+02	5.23E+02	3.57E+02	0	0
	Mean	4.03E+02	4.11E+02	3.98E+02	4.20E+02	3.94E+02	1.61E+03	4.02E+02	0	0
	Var	5.66E+02	3.72E+02	6.71E+02	1.14E+02	5.93E+02	2.18E+06	5.39E+02	0	0
	Dist	6.18E+01	6.43E+01	6.05E+01	6.41E+01	5.95E+01	9.71E+01	6.24E+01	0	0
f_7	Best	4.05E+02	4.35E+02	4.16E+02	5.03E+02	4.58E+02	4.33E+02	4.82E+02	0	0
	Mean	5.33E+02	5.22E+02	5.27E+02	5.36E+02	5.47E+02	3.47E+03	5.57E+02	0	0
	Var	3.44E+03	2.23E+03	2.71E+03	3.17E+02	1.99E+03	7.09E+06	2.50E+03	0	0
	Dist	6.61E+03	6.56E+03	6.59E+03	6.52E+03	6.66E+03	1.12E+04	6.79E+03	0	0
f_8	Best	4.79E+01	4.84E+01	5.14E+01	5.76E+01	4.41E+01	5.45E+01	4.68E+01	0	0
	Mean	5.75E+01	5.85E+01	5.91E+01	6.41E+01	5.49E+01	8.98E+01	5.57E+01	0	0
	Var	2.35E+01	1.89E+01	1.32E+01	6.69	1.67E+01	6.19E+02	2.38E+01	0	0
	Dist	1.27E+02	1.28E+02	1.26E+02	1.20E+02	1.23E+02	2.14E+02	1.24E+02	0	0
f_9	Best	8.49E+03	9.36E+03	8.69E+03	9.39E+03	8.53E+03	-4.74E+34	9.49E+03	1.61E+03	3.55
	Mean	9.73E+03	9.72E+03	9.63E+03	9.98E+03	9.39E+03	-3.36E+33	1.00E+04	1.61E+03	3.55
	Var	1.72E+05	8.37E+04	1.57E+05	5.48E+04	1.25E+05	1.21E+68	1.00E+05	4.90E-25	0
	Dist	1.11E+04	1.16E+04	1.17E+04	1.20E+04	1.13E+04	4.34E+33	1.33E+04	6.29E+02	2.91E+01



(a) Uniform initialization (b) SDI (c) SDI_opt (d) DLU

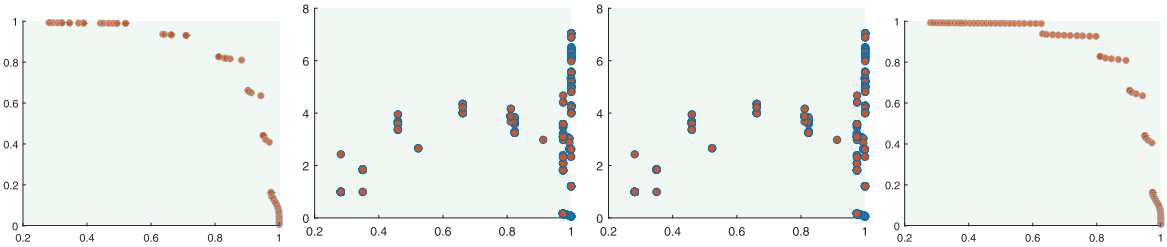
FIGURE 11. Non-dominated front with MOEA/D on ZDT3.



(a) Uniform initialization (b) SDI (c) SDI_opt (d) DLU

FIGURE 12. Non-dominated front with MOEA/D on ZDT4.

it is unrealistic to test all these functions due to the length of the paper. Therefore, we choose some representative functions with different properties. The 10 selected benchmarks, shown in Table 12 are all minimization problems



(a) Uniform initialization (b) SDI (c) SDI_opt (d) DLU

FIGURE 13. Non-dominated front with MOEA/D on ZDT6.

TABLE 9. Friedman test of different initialization methods.

Fun	p	Random	Uniform	LHS	DUD	OLM	Chaotic1	Chaotic2	SDI	DLU
DE-a	$\ll$ 0.001	5.431	5.292	6.236	5.653	6.125	4.972	6.458	2.486	2.347
PSO-w	$\ll$ 0.001	5.764	5.847	5.653	5.181	4.375	6.917	6.431	2.458	2.375
CS	$\ll$ 0.001	5.445	5.903	5.806	5.889	4.250	5.333	7.541	2.486	2.347
ABC	$\ll$ 0.001	5.639	5.722	5.417	5.472	4.81	7.722	6.722	1.833	1.667

in the $[-100, 100]^D$ regular domain (D is the dimension of the problem), which consists of 6 CEC2014 problems and 4 CEC2017 problems.

Taking the cuckoo search as an example, we observe whether our initialization method is still valid in the case of 30D. The parameter setting of CS is the same as that in Sec. IV. For the SDI, q is still set to 11, which means that the population size is 3,311, and when the number of maximum iterations is 60, the maximum fitness evaluations (FEs) is 198,660. For a fair comparison, all approaches should have the same or at least similar termination conditions. Hence, for other initialization methods (including DLU), the population size is set to 100, the number of iterations is 1,980, that is, the termination condition: the maximum fitness evaluations is 198,000. Each initialization technique runs independently 20 times.

The experimental results are shown in Table 13. It can be seen that for all test functions, the corresponding results (whether the 'Best' or the 'Mean') of DLU are better than other methods. Although SDI has 660 times more fitness estimations than other comparison methods, the final solution accuracy is not better than others. In addition, we find that the variance of our method is smaller in most cases, which indicates that our initialization method improves the robustness of the algorithm.

The Friedman test was undertaken, as shown in Table 14. It was found that the proposed DLU was ranked first. The p value is far less than 0.05, indicating that there are significant differences among these nine initialization methods. This means our proposed method is still effective for more complex problems.

B. LOCAL DIVERSITY

Population diversity is a key measurement in metaheuristic optimizers. Traditional diversity celebrates the success of assessing the quality of the population in the whole/global space. It is generally believed that higher diversity means more detailed exploration of the black-box problems. Regularly, researchers strongly prefer to choose the initial population with much higher diversity to avoid the lack of local information from the complex functions. However, after several iterations, the population gradually converges to

TABLE 10. Comparisons of different initialization methods for multiobjective problems.

Fun		Uniform	LHS	SDI-opt	DLU
ZDT1	Mean	1.49E-02	1.49E-02	1.47E-02	1.49E-02
	Var	9.15E-05	8.84E-05	8.51E-05	9.36E-05
	γ	5.17E-01	4.50E-01	2.85E-01	3.73E-01
	σ	3.78E-01	3.41E-01	3.10E-01	3.41E-01
ZDT2	Mean	1.52E-02	1.63E-02	4.33E-02	1.48E-02
	Var	2.16E-05	5.56E-05	1.40E-03	9.77E-06
	γ	6.21E-01	6.28E-01	5.28	5.76E-01
	σ	2.29E-01	2.91E-01	1.12	1.79E-01
ZDT3	Mean	3.25E-02	3.12E-02	2.43E-02	2.26E-02
	Var	2.40E-03	1.30E-03	8.79E-04	8.36E-04
	γ	1.69E+01	3.11E+01	1.51E+01	1.23E+01
	σ	1.48	1.52	1.32	1.30
ZDT4	Mean	2.00E-02	1.88E-02	1.48E-02	1.48E-02
	Var	3.04E-04	2.16E-04	8.31E-05	7.84E-05
	γ	3.47	2.70	0.30	0.29
	σ	6.94E-01	6.81E-01	3.21E-01	3.23E-01
ZDT6	Mean	2.00E-02	2.31E-02	2.35E-02	2.29E-02
	Var	2.55E-02	1.70E-03	1.70E-03	1.60E-03
	γ	1.43E+01	1.46E+01	1.45E+01	1.45E+01
	σ	1.19	1.11	1.13	1.11

TABLE 11. Friedman test of different initialization methods.

Fun	p	Uniform	LHS	SDI-opt	DLU
ZDT1	0.025	3.50	2.50	1	3
ZDT2	0.007	2	3	4	1
ZDT3	0.001	3.50	3.50	2	1
ZDT4	0.010	4	3	1.625	1.375
ZDT6	0.288	2.50	2.875	3.125	1.50
mean	0.0004	3.10	2.975	2.35	1.575

TABLE 12. The ten test functions in CEC2014 and CEC2017.

Type	No.	Function	Opt
Unimodal	f_2	Rotated Bent Cigar Function	200
Multimodal	f_8	Shifted Rastrigin's Function	800
	f_{10}	Shifted Schwefel's Function	1000
	f_{16}	Shifted and Rotated Expanded Scaffer's F6 Function	1600
Hybrid	f_{20}	Hybrid Function 4 (N=4)	2000
Composition	f_{26}	Composition Function 4 (N=5)	2600
Multimodal	F_3	Shifted and Rotated Zakharov function	300
Composition	F_{22}	Composition Function 2 (N=3)	2200
	F_{24}	Composition Function 4 (N=4)	2400
	F_{26}	Composition Function 7 (N=6)	2600

TABLE 13. Comparisons of different initialization methods with CS (30D) on CEC2014 and CEC2017.

Fun	Value	Random	Uniform	LHS	DUD	OLM	Chaotic1	Chaotic2	SDI	DLU
f_2	Best	4.89E+02	5.41E+02	5.60E+02	5.24E+02	4.43E+02	3.77E+02	4.80E+02	1.48E+10	4.11E+02
	Mean	6.72E+02	6.85E+02	6.97E+02	6.91E+02	6.48E+02	6.98E+02	6.22E+02	2.15E+10	6.01E+02
	Var	2.13E+04	1.19E+04	1.44E+04	1.38E+04	1.87E+04	3.53E+04	1.21E+04	7.11E+18	1.29E+04
f_8	Best	8.6EE+02	8.66E+02	8.60E+02	8.64E+02	8.63E+02	8.61E+02	8.62E+02	1.02E+03	8.47E+02
	Mean	8.78E+02	8.81E+02	8.79E+02	8.77E+02	8.79E+02	8.78E+02	8.76E+02	1.03E+03	8.66E+02
	Var	6.68E+01	6.47E+01	8.92E+01	5.38E+01	5.34E+01	6.69E+01	4.46E+01	5.06E+01	4.79E+01
f_{10}	Best	2.53E+03	2.57E+03	2.70E+03	2.69E+03	2.41E+03	2.56E+03	2.83E+03	5.22E+02	1.49E+02
	Mean	2.97E+03	2.94E+03	2.97E+03	2.94E+03	2.90E+03	2.92E+03	3.16E+03	5.47E+03	1.76E+03
	Var	4.38E+04	3.43E+04	2.33E+04	1.64E+04	5.15E+03	2.85E+04	2.75E+04	1.68E+04	1.75E+04
f_{16}	Best	1.61E+03	1.61E+03	1.61E+03	1.61E+03	1.61E+03	1.61E+03	1.61E+03	1.61E+03	1.61E+03
	Mean	1.61E+03	1.61E+03	1.61E+03	1.61E+03	1.61E+03	1.61E+03	1.61E+03	1.61E+03	1.61E+03
	Var	5.12E-02	3.83E-02	6.57E-02	3.92E-02	3.42E-02	4.10E-02	3.06E-02	2.87E-02	4.31E-02
f_{20}	Best	2.09E+03	2.10E+03	2.08E+03	2.08E+03	2.10E+03	2.09E+03	2.07E+03	3.62E+03	2.08E+03
	Mean	2.12E+03	2.12E+03	2.12E+03	2.12E+03	2.12E+03	2.13E+03	2.12E+03	1.46E+04	2.12E+03
	Var	1.79E+02	2.29E+02	3.67E+02	4.14E+02	1.05E+02	3.42E+02	4.43E+02	1.44E+07	2.12E+02
f_{26}	best	2.70E+03	2.70E+03	2.70E+03	2.70E+03	2.70E+03	2.70E+03	2.70E+03	2.70E+03	2.70E+03
	Mean	2.70E+03	2.70E+03	2.70E+03	2.70E+03	2.70E+03	2.70E+03	2.70E+03	2.70E+03	2.70E+03
	Var	1.43E-03	1.93E-03	1.29E-03	5.04E-04	1.33E-03	9.32E-04	9.19E-04	2.75E-02	8.78E-04
F_3	Best	4.66E+04	3.31E+04	3.97E+04	4.36E+04	3.98E+04	3.48E+04	3.85E+04	5.77E+04	1.45E+04
	Mean	5.60E+04	5.71E+04	5.55E+04	6.12E+04	5.20E+04	5.32E+04	5.74E+04	6.74E+04	3.20E+04
	Var	3.43E+07	9.00E+07	6.79E+07	6.93E+07	4.36E+07	7.63E+07	1.03E+08	1.62E+07	6.92E+07
F_{22}	Best	2.32E+03	2.33E+03	2.34E+03	2.35E+03	2.32E+03	2.32E+03	23.32E+03	4.33E+03	2.30E+03
	Mean	2.85E+03	2.79E+03	2.86E+03	2.49E+03	2.55E+03	2.56E+03	3.40E+03	4.95E+03	2.30E+03
	Var	1.41E+06	1.43E+06	1.36E+06	4.18E+04	7.73E+05	2.27E+05	2.12E+06	1.21E+05	8.63
F_{24}	Best	2.72E+03	2.71E+03	2.68E+03	2.71E+03	2.92E+03	2.74E+03	2.76E+03	3.12E+03	2.54E+03
	Mean	2.95E+03	2.92E+03	2.94E+03	2.94E+03	2.95E+03	2.94E+03	2.96E+03	3.15E+03	2.57E+03
	Var	3.30E+03	8.34E+03	6.00E+03	6.20E+03	4.11E+02	4.78E+03	2.52E+03	2.68E+02	8.40E+02
F_{26}	Best	3.21E+03	3.11E+03	3.19E+03	3.13E+03	3.11E+03	3.04E+03	3.16E+03	5.13E+03	2.92E+03
	Mean	3.66E+03	3.53E+03	3.50E+03	3.36E+03	3.46E+03	3.47E+03	3.80E+03	6.09E+03	3.09E+03
	Var	1.88E+05	6.67E+04	4.22E+04	3.10E+04	5.96E+04	4.62E+04	3.43E+05	1.12E+05	2.19E+04

TABLE 14. Friedman test of different initialization methods.

p	Random	Uniform	LHS	DUD	OLM	Chaotic1	Chaotic2	SDI	DLU
$\ll$ 0.001	5.32	5.45	5.23	4.93	4.68	5.07	5.23	7.03	2.05

the optimum, and the population diversity will continue to decrease. At this time, high diversity means that the algorithm can effectively avoid premature convergence, but it can also mean slow convergence. On the other hand, if the diversity is low (for example, the diversity is 0), the population may converge to the optimum, but it may also fall into the local optimum. This suggests that the previous diversity index is more effective for the initial state, but not suitable for measuring the population quality in the iterative state.

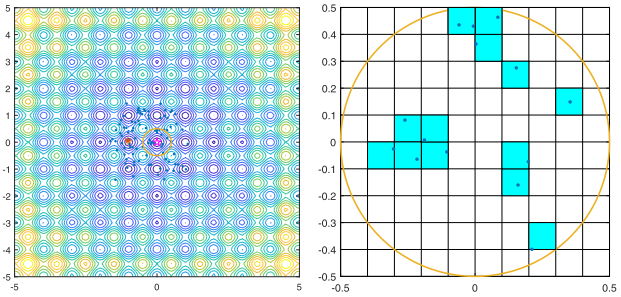
Therefore, in the process of iterations, we should consider the relationship between population diversity and convergence [69]. We hope that more individuals are close to the optimal solution, and the diversity of the population near the optimum is better. Hence, we innovatively proposed a local diversity measure, a.k.a L-diversity. L-diversity focuses on the local-space near the global optimal points OPT. At any iterations, we pay close attention to points set $P_r = \{p_i | dis_{p_i,OPT} < r\}$, here $dis_{p_i,OPT}$ is the distance between the point p_i and OPT, r is the constant distance. To measure the local density, we divided the local space into m parts equally. The part's domination means that at least one point falls into the corresponding part. Clearly, in the local space around the OPT point, more parts dominated means more detailed 'useful' information achieved in evolutionary computation. Theoretically, the L-diversity is to quantify the amount of local dominated parts.

Take the Rastriginas function as an example, the L-diversity is described in detail as follows. By the way, it can prove the difference between DLU and Random initialization after several iterations. As shown in Fig. 14 and Fig. 15, the optimal solution is located at $(0, 0)$, and the radius r is assumed to be 1. After two iterations of PSO, 29 points of the initial population generated by DLU are located in the neighborhood of the optimum, while the initial population with random distribution can only make 14 points in the neighborhood of the optimal solution after two iterations. These points are what we need to pay attention to. Next, we divide the circular neighborhood of the global optimum into a grid with a side length of 0.1. The blue squares indicate that at least one individual falls in the grid, that is, the grid is dominated (covered). It can be seen that the number of grids covered by DLU is much larger than that of the random method. It can verify that the DLU can make the algorithm converge to the optimal solution quickly.

In fact, the vicinity of the optimal solution contains more useful information. DLU helps to extract more useful information faster. Therefore, DLU method can make the algorithm have better L-diversity in subsequent iterations.

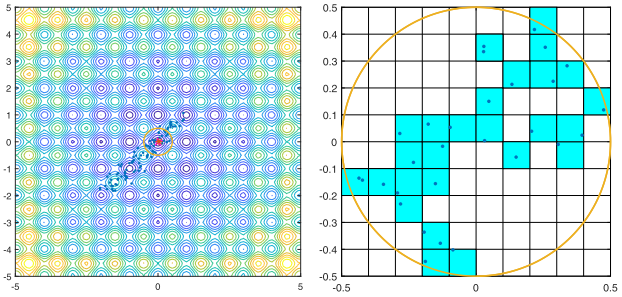
C. DISCUSSION AND ANALYSIS

Some steps of the metaheuristic algorithms, such as update and selection, have attracted the attention of many



(a) After 2 iterations of PSO. (b) Distribution state in neighborhood of optimal

FIGURE 14. The initial population obeys random distribution.



(a) After 2 iterations of PSO. (b) Distribution state in neighborhood of optimal

FIGURE 15. The initial population generated by DLU.

researchers. Based on these, a large number of new algorithms [70] and modified algorithms have been proposed. Initialization is a crucial step that is often ignored. There has been a lack of in-depth research on initialization. In fact, choosing a better initial population usually makes the subsequent search process more effective. Hence, it is very meaningful if we can do some work in this area to improve the performance of algorithms.

In this paper, we proposed a novel initialization technique for metaheuristic optimizers, called diagonal linear uniform initialization (DLU). The DLU is simple and easy to implement. It carries the information of each dimension uniformly, and only needs a small number of the initial population. Experiments show that this method can improve the performance of most optimizers. The most important thing is that DLU still performs well when the dimensionality of the problem increases. This is crucial for dealing with large-scale problems.

We give a new view of the search space in the regular domain, which will be a simple step for such an open problem. We hope these excited conclusions from our work can be meaningful for the metaheuristic algorithm research. We will focus on the statistical characteristics in the iterative process of the algorithm and its relationship with the initial population. Hope that in the future, an adaptive initialization method will be given for the problem and the given algorithm. Of course, this requires joint efforts in the fields of statistics, optimization and intelligent algorithms.

VI. CONCLUSION

Initialization, as an essential step in metaheuristic optimizers, seriously affects the convergence speed and accuracy of the algorithm. In the past, some initialization strategies were proposed gradually, but there are still some problems: the curse of dimensionality; high computational cost; sensitive to parameters. Many studies paying too much attention to the uniformity of the initial distribution, without considering the update and selection mechanism of the algorithm. Metaheuristic optimizers need a simpler and more effective strategy to provide a better initial population.

This paper proposed a new initialization strategy called diagonal linear uniform initialization. The entire space is divided into multiple unit-spaces based on the grid, and selects some vertices of the diagonal subspace as the initial population. A small population carries a large amount of information in each dimension, which can guide the search process well. Another advantage of DLU is that its performance remains valid when the dimensionality of the problem increases. Benchmark functions on 10-dimensional, 30-dimensional and 100-dimensional were tested. Experimental results show that the accuracy of the solutions obtained by DE through DLU initialization is higher than that of random, uniform, LHS, DUD, OLM, Chaotic1, Chaotic2, and SDI. Under the same conditions, extended experiments show that DLU is still effective for three swarm intelligence algorithms: PSO, CS and ABC. Finally, the proposed method also shows better performance when solving five standard multi-objective problems.

Although the proposed strategy was a good choice for initialization, the information for solving problems has not been taken into account yet. Obviously, it is not proper to use the same initial method for different problems. Further investigations in this area will be part of our future research. Also, all our tests have used the benchmarks with simple bounds on regular domains. It is necessary to test more complicated issues with nonlinear constraints on irregular search domains to see if the same conclusions still hold. Such problems can be drawn from real-world applications.

REFERENCES

[1] X. Li, S. Ma, and Y. Wang, "Multi-population based ensemble mutation method for single objective bilevel optimization problem," *IEEE Access*, vol. 4, pp. 7262–7274, 2016, doi: 10.1109/ACCESS.2016.2617738.

[2] H. Hakli and M. S. Kiran, "An improved artificial bee colony algorithm for balancing local and global search behaviors in continuous optimization," *Int. J. Mach. Learn. Cybern.*, vol. 11, no. 9, pp. 2051–2076, 2020, doi: 10.1007/s13042-020-01094-7.

[3] C. Dong, Z. Xiong, X. Liu, Y. Ye, Y. Yang, and W. Guo, "Dual-search artificial bee colony algorithm for engineering optimization," *IEEE Access*, vol. 7, pp. 24571–24584, 2019, doi: 10.1109/ACCESS.2019.2899743.

[4] A. Azizi, "A case study on computer-based analysis of the stochastic stability of mechanical structures driven by white and colored noise: Utilizing artificial intelligence techniques to design an effective active suspension system," *Complexity*, vol. 2020, Apr. 2020, Art. no. 7179801, doi: 10.1155/2020/7179801.

[5] R. Alipour-Sarabi, Z. Nasiri-Gheidari, F. Tootoonchian, and H. Oraee, "Improved winding proposal for wound rotor resolver using genetic algorithm and winding function approach," *IEEE Trans. Ind. Electron.*, vol. 66, no. 2, pp. 1325–1334, Feb. 2019, doi: 10.1109/TIE.2018.2821091.

[6] M. F. Zaman, S. M. Elsayed, T. Ray, and R. A. Sarker, "Evolutionary algorithms for dynamic economic dispatch problems," *IEEE Trans. Power Syst.*, vol. 31, no. 2, pp. 1486–1495, Mar. 2016, doi: 10.1109/TPWRS.2015.2428714.

[7] S. Mishra, C. Hota, L. Kumar, and A. Nayak, "An evolutionary GA-based approach for community detection in IoT," *IEEE Access*, vol. 7, pp. 100512–100534, 2019, doi: 10.1109/ACCESS.2019.2923965.

[8] Q. Li, S.-Y. Liu, and X.-S. Yang, "Neighborhood information-based probabilistic algorithm for network disintegration," *Expert Syst. Appl.*, vol. 139, Jan. 2020, Art. no. 112853, doi: 10.1016/j.eswa.2019.112853.

[9] L. Sun, L. Lin, M. Gen, and H. Li, "A hybrid cooperative coevolution algorithm for fuzzy flexible job shop scheduling," *IEEE Trans. Fuzzy Syst.*, vol. 27, no. 5, pp. 1008–1022, May 2019, doi: 10.1109/TFUZZ.2019.2895562.

[10] X. Sun, S. Guo, J. Guo, and B. Du, "A hybrid multi-objective evolutionary algorithm with heuristic adjustment strategies and variable neighborhood search for flexible job-shop scheduling problem considering flexible rest time," *IEEE Access*, vol. 7, pp. 157003–157018, 2019, doi: 10.1109/ACCESS.2019.2948057.

[11] K. Gao, F. Yang, M. Zhou, Q. Pan, and P. N. Suganthan, "Flexible job-shop rescheduling for new job insertion by using discrete jaya algorithm," *IEEE Trans. Cybern.*, vol. 49, no. 5, pp. 1944–1955, May 2019, doi: 10.1109/TCYB.2018.2817240.

[12] J. Liu, Y. Chi, S. He, and Z. Liu, "Ensemble multi-objective evolutionary algorithm for gene regulatory network reconstruction based on fuzzy cognitive maps," *CAAI Trans. Intell. Technol.*, vol. 4, no. 1, pp. 24–36, 2019, doi: 10.1049/trit.2018.1059.

[13] S. Lalwani, R. Kumar, H. Sharma, and A. Verma, "Efficient discrete firefly algorithm for Ctrie based caching of multiple sequence alignment on optimally scheduled parallel machines," *CAAI Trans. Intell. Technol.*, vol. 4, no. 2, pp. 92–100, 2019, doi: 10.1049/trit.2018.1040.

[14] T. X. Pham, P. Siarry, and H. Oulhadj, "Segmentation of MR brain images through hidden Markov random field and hybrid Metaheuristic algorithm," *IEEE Trans. Image Process.*, vol. 29, pp. 6507–6522, 2020, doi: 10.1109/TIP.2020.2990346.

[15] S. Mirjalili, S. M. Mirjalili, and A. Lewis, "Grey wolf optimizer," *Adv. Eng. Softw.*, vol. 69, pp. 46–61, Mar. 2014, doi: 10.1016/j.advengsoft.2013.12.007.

[16] M. H. Nadimi-Shahraki, S. Taghian, and S. Mirjalili, "An improved grey wolf optimizer for solving engineering problems," *Expert Syst. Appl.*, vol. 166, Mar. 2021, Art. no. 113917.

[17] I. Aljarah, H. Faris, and S. Mirjalili, "Optimizing connection weights in neural networks using the whale optimization algorithm," *Soft Comput.*, vol. 22, no. 1, pp. 1–15, 2018, doi: 10.1007/s00500-016-2442-1.

[18] R. Salgotra and U. Singh, "The naked mole-rat algorithm," *Neural Comput. Appl.*, vol. 31, no. 12, pp. 8837–8857, 2019, doi: 10.1007/s00521-019-04464-7.

[19] X. Li, J. Zhang, and M. Yin, "Animal migration optimization: An optimization algorithm inspired by animal migration behavior," *Neural Comput. Appl.*, vol. 24, no. 7, pp. 1867–1877, 2014, doi: 10.1007/s00521-013-1433-8.

[20] X.-S. Yang, "Swarm intelligence based algorithms: A critical analysis," *Evol. Intell.*, vol. 7, no. 1, pp. 17–28, 2014, doi: 10.1007/s12065-013-0102-2.

[21] S. Kondamadugula and S. R. Naidu, "Accelerated evolutionary algorithms with parameterimportance based population initialization for variation-aware analog yield optimization," in *Proc. IEEE 59th Int. Midwest Symp. Circuits Syst. (MWSCAS)*, Oct. 2016, pp. 1–4, doi: 10.1109/MWSCAS.2016.7869996.

[22] B. Kazimipour, X. Li, and A. K. Qin, "A review of population initialization techniques for evolutionary algorithms," in *Proc. IEEE Congr. Evol. Comput. (CEC)*, Jul. 2014, pp. 2585–2592, doi: 10.1109/CEC.2014.6900618.

[23] H. Maaranen, K. Miettinen, and A. Penttinen, "On initial populations of a genetic algorithm for continuous optimization problems," *J. Global Optim.*, vol. 37, no. 3, p. 405, 2007, doi: 10.1007/s10898-006-9056-6.

[24] S. Kimura and K. Matsumura, "Genetic algorithms using low-discrepancy sequences," in *Proc. Conf. Genetic Evol. Comput. (GECCO)*, 2005, pp. 1341–1346.

[25] Z. Ma and G. A. E. Vandenbosch, "Impact of random number generators on the performance of particle swarm optimization in antenna design," in *Proc. 6th Eur. Conf. Antennas Propag. (EUCAP)*, Mar. 2012, pp. 925–929, doi: 10.1109/EuCAP.2012.6205998.

[26] A. Azizi, "Introducing a novel hybrid artificial intelligence algorithm to optimize network of industrial applications in modern manufacturing," *Complexity*, vol. 2017, Jun. 2017, Art. no. 8728209.

[27] L. Peng, Y. Wang, G. Dai, and Z. Cao, "A novel differential evolution with uniform design for continuous global optimization," *J. Comput.*, vol. 7, no. 1, pp. 3–10, 2012, doi: 10.4304/jcp.7.1.3-10.

[28] K. T. Fang and D. K. J. Lin, "Uniform experimental designs and their applications in industry," in *Handbook of Statistics*, vol. 22, no. 3. 2003, ch. 4, pp. 131–170, doi: 10.1016/S0169–7161(03)22006-X.

[29] C. Ran, M. Li, L. Ke, and Y. Xin, "Evolutionary multiobjective optimization-based multimodal optimization: Fitness landscape approximation and peak detection," *IEEE Trans. Evol. Comput.*, vol. 22, no. 5, pp. 692–706, Oct. 2018, doi: 10.1109/TEVC.2017.2744328.

[30] W.-F. Gao and S.-Y. Liu, "A modified artificial bee colony algorithm," *Comput. Oper. Res.*, vol. 39, no. 3, pp. 687–697, Mar. 2012, doi: 10.1016/j.cor.2011.06.007.

[31] B. Alatas, "Chaotic bee colony algorithms for global numerical optimization," *Expert Syst. Appl.*, vol. 37, no. 8, pp. 5682–5687, Aug. 2010, doi: 10.1016/j.eswa.2010.02.042.

[32] S. Rahnamayan, H. R. Tizhoosh, and M. M. A. Salama, "Opposition-based differential evolution," *IEEE Trans. Evol. Comput.*, vol. 12, no. 1, pp. 64–79, Feb. 2008, doi: 10.1109/TEVC.2007.894200.

[33] S. Elsayed, R. Sarker, and C. A. Coello Coello, "Sequence-based deterministic initialization for evolutionary algorithms," *IEEE Trans. Cybern.*, vol. 47, no. 9, pp. 2911–2923, Sep. 2017, doi: 10.1109/tcyb.2016.2630722.

[34] V. V. de Melo and A. C. Botazzo Delbem, "Investigating smart sampling as a population initialization method for differential evolution in continuous problems," *Inf. Sci.*, vol. 193, pp. 36–53, Jun. 2012, doi: 10.1016/j.ins.2011.12.037.

[35] B. Kazimipour, X. Li, and A. K. Qin, "Initialization methods for large scale global optimization," in *Proc. IEEE Congr. Evol. Comput.*, Jun. 2013, pp. 2750–2757, doi: 10.1109/CEC.2013.6557902.

[36] B. Liu, L. Wang, Y.-H. Jin, F. Tang, and D.-X. Huang, "Improved particle swarm optimization combined with chaos," *Chaos, Solitons Fractals*, vol. 25, no. 5, pp. 1261–1271, Sep. 2005, doi: 10.1016/j.chaos.2004.11.095.

[37] M. Gong, L. Jiao, F. Liu, and W. Ma, "Immune algorithm with orthogonal design based initialization, cloning, and selection for global optimization," *Knowl. Inf. Syst.*, vol. 25, no. 3, pp. 523–549, 2010, doi: 10.1007/s10115-009-0261-8.

[38] G. Zhang, L. Gao, and Y. Shi, "An effective genetic algorithm for the flexible job-shop scheduling problem," *Expert Syst. Appl.*, vol. 38, no. 4, pp. 3563–3573, Apr. 2011, doi: 10.1016/j.eswa.2010.08.145.

[39] C. Li, X. Chu, Y. Chen, and L. Xing, "A knowledge-based initialization technique of genetic algorithm for the travelling salesman problem," in *Proc. 11th Int. Conf. Natural Comput. (ICNC)*, Aug. 2015, pp. 188–193.

[40] T. Chang, D. Kong, N. Hao, K. Xu, and G. Yang, "Solving the dynamic weapon target assignment problem by an improved artificial bee colony algorithm with heuristic factor initialization," *Appl. Soft Comput.*, vol. 70, pp. 845–863, Sep. 2018, doi: 10.1016/j.asoc.2018.06.014.

[41] S. S. Politis, Z. Zhang, Z. Han, J. J. Hasenbein, and M. Arellano, "Stochastic analysis of network-level bridge maintenance needs using latin hypercube sampling," *ASCE-ASME J. Risk Uncertainty Eng. Syst., A: Civil Eng.*, vol. 7, no. 1, Mar. 2021, Art. no. 04020049, doi: 10.1061/AJRUA6.0001099.

[42] Q. Zhang and H. Li, "MOEA/D: A multiobjective evolutionary algorithm based on decomposition," *IEEE Trans. Evol. Comput.*, vol. 11, no. 6, pp. 712–731, Dec. 2007, doi: 10.1109/TEVC.2007.892759.

[43] G. Heidari-Bateni and C. D. McGillem, "A chaotic direct-sequence spread-spectrum communication system," *IEEE Trans. Commun.*, vol. 42, nos. 2–4, pp. 1524–1527, Feb. 1994, doi: 10.1109/TCOMM.1994.582834.

[44] S. Arora, M. Sharma, and P. Anand, "A novel chaotic interior search algorithm for global optimization and feature selection," *Appl. Artif. Intell.*, vol. 34, no. 4, pp. 292–328, 2020, doi: 10.1080/08839514.2020.1712788.

[45] T. B. Nkwanyana and Z. Wang, "Improved particle swarm optimization base on the combination of linear decreasing and chaotic inertia weights," in *Proc. 12th Int. Conf. Comput. Intell. Commun. Netw. (CICN)*, Sep. 2020, pp. 460–465, doi: 10.1109/CICN49253.2020.9242603.

[46] L. D. S. Coelho and V. C. Mariani, "Use of chaotic sequences in a biologically inspired algorithm for engineering design optimization," *Expert Syst. Appl.*, vol. 34, no. 3, pp. 1905–1913, Apr. 2008, doi: 10.1016/j.eswa.2007.02.002.
[47] C.-H. Chou and J.-N. Chen, "Genetic algorithms: Initialization schemes and genes extraction," in *Proc. 9th IEEE Int. Conf. Fuzzy Syst. (FUZZ)*, vol. 2, May 2000, pp. 965–968, doi: 10.1109/FUZZY.2000.839167.
[48] M. D. McKay, R. J. Beckman, and W. J. Conover, "Comparison of three methods for selecting values of input variables in the analysis of output from a computer code," *Technometrics*, vol. 21, no. 2, pp. 239–245, May 1979, doi: 10.1080/00401706.1979.10489755.
[49] Q. Li, S.-Y. Liu, and X.-S. Yang, "Influence of initialization on the performance of Metaheuristic optimizers," *Appl. Soft Comput.*, vol. 91, Jun. 2020, Art. no. 106193, doi: 10.1016/j.asoc.2020.106193.
[50] J. C. Helton and F. J. Davis, "Latin hypercube sampling and the propagation of uncertainty in analyses of complex systems," *Rel. Eng. Syst. Saf.*, vol. 81, no. 1, pp. 23–69, 2003, doi: 10.1016/S0951-8320(03)00058-9.
[51] W. Yuan and K. T. Fang, "A note on uniform distribution and experimental design," *A Monthly J. Sci.*, vol. 26, no. 6, p. 485, 1981, doi: 10.1142/9789812701190_0035.
[52] H. T. Rauf, U. Shoaib, M. I. Lali, M. Alhaisoni, M. N. Irfan, and M. A. Khan, "Particle swarm optimization with probability sequence for global optimization," *IEEE Access*, vol. 8, pp. 110535–110549, 2020, doi: 10.1109/ACCESS.2020.3002725.
[53] X. S. Yang, *Nature-Inspired Optimization Algorithms*. Pittsburgh, PA, USA: Academic, 2020.
[54] X.-S. Yang and S. Deb, "Cuckoo Search via Lévy flights," in *Proc. World Congr. Nature Biol. Inspired Comput. (NaBIC)*, Dec. 2009, pp. 210–214, doi: 10.1109/NABIC.2009.5393690.
[55] D. Karaboga and B. Akay, "A comparative study of artificial bee colony algorithm," *Appl. Math. Comput.*, vol. 214, no. 1, pp. 108–132, Aug. 2009, doi: 10.1016/j.amc.2009.03.090.
[56] X. Li, S. Ma, and J. Hu, "Multi-search differential evolution algorithm," *Appl. Intell.*, vol. 47, no. 1, pp. 231–256, 2017, doi: 10.1007/s10489-016-0885-9.
[57] H. Zhou and X. Wei, "Particle swarm optimization based on a novel evaluation of diversity," *Algorithms*, vol. 14, no. 2, p. 29, Jan. 2021, doi: 10.3390/a14020029.
[58] X. Li and M. Yin, "Modified cuckoo search algorithm with self adaptive parameter method," *Inf. Sci.*, vol. 298, pp. 80–97, Mar. 2015, doi: 10.1016/j.ins.2014.11.042.
[59] J.-Q. Li and Y.-Q. Han, "A hybrid multi-objective artificial bee colony algorithm for flexible task scheduling problems in cloud computing system," *Cluster Comput.*, vol. 23, no. 4, pp. 2483–2499, 2020, doi: 10.1007/s10586-019-03022-z.
[60] H. Du, Z. Wang, W. Zhan, and J. Guo, "Elitism and distance strategy for selection of evolutionary algorithms," *IEEE Access*, vol. 6, pp. 44531–44541, 2018, doi: 10.1109/ACCESS.2018.2861760.
[61] Y. Shi and R. Eberhart, "A modified particle swarm optimizer," in *Proc. IEEE Int. Conf. Evol. Comput. IEEE World Congr. Comput. Intell.*, May 1998, pp. 69–73, doi: 10.1109/ICEC.1998.699146.
[62] Q. Fan, X. Yan, and Y. Zhang, "Auto-selection mechanism of differential evolution algorithm variants and its application," *Eur. J. Oper. Res.*, vol. 270, no. 2, pp. 636–653, Oct. 2018, doi: 10.1016/j.ejor.2017.10.013.
[63] E. J. Hughes, "Evolutionary many-objective optimisation: Many once or one many?" in *Proc. IEEE Congr. Evol. Comput.*, vol. 1, Sep. 2005, pp. 222–227.
[64] V. A. Shim, K. C. Tan, and C. Y. Cheong, "A hybrid estimation of distribution algorithm with decomposition for solving the multiobjective multiple traveling salesman problem," *IEEE Trans. Syst., Man, Cybern., C (Appl. Rev.)*, vol. 42, no. 5, pp. 682–691, Sep. 2012, doi: 10.1109/TSMCC.2012.2188285.
[65] Y. Mei, K. Tang, and X. Yao, "Decomposition-based memetic algorithm for multiobjective capacitated arc routing problem," *IEEE Trans. Evol. Comput.*, vol. 15, no. 2, pp. 151–165, Apr. 2011, doi: 10.1109/TEVC.2010.2051446.
[66] S. Pal, S. Das, A. Basak, and P. N. Suganthan, "Synthesis of difference patterns for monopulse antennas with optimal combination of array-size and number of subarrays—A multi-objective optimization approach," *Prog. Electromagn. Res. B*, vol. 21, no. 3, pp. 257–280, 2010, doi: 10.1590/0001-3765201520140332.
[67] L. Ke, Q. Zhang, and R. Battiti, "Hybridization of decomposition and local search for multiobjective optimization," *IEEE Trans. Cybern.*, vol. 44, no. 10, pp. 1808–1820, Oct. 2014, doi: 10.1109/TCYB.2013.2295886.
[68] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, Apr. 2002, doi: 10.1109/4235.996017.
[69] S. Cheng, Y. Shi, Q. Qin, Q. Zhang, and R. Bai, "Population diversity maintenance in brain storm optimization algorithm," *J. Artif. Intell. Soft Comput. Res.*, vol. 4, no. 2, pp. 83–97, 2014, doi: 10.1515/jaiscr-2015-0001.
[70] A. Azizi, A. Vatankhah Barenji, and M. Hashmipour, "Optimizing radio frequency identification network planning through ring probabilistic logic neurons," *Adv. Mech. Eng.*, vol. 8, no. 8, 2016, Art. no. 1687814016663476, doi: 10.1177/1687814016663476.



QIAN LI received the M.S. degree in applied mathematics, in 2017. She is currently pursuing the Ph.D. degree with the School of Mathematics and Statistics, Xidian University, Xi'an, China. Her research interests include swarm and evolutionary algorithms, algorithmic application, and complex network structure.



YIGUANG BAI received the B.S. degree in mathematics and applied mathematics, and the M.S. degree in applied mathematics from Xidian University, Xi'an, China, in 2014 and 2017, respectively, where he is currently pursuing the Ph.D. degree with the School of Mathematics and Statistics. He is also a Visiting Student studying with the Cheriton School of Computer Science, University of Waterloo, Waterloo, ON, Canada, for a period one year. His research interests include optimization methods and their applications in large-scale network structure, robustness and function.



WEIFENG GAO received the B.S. and Ph.D. degrees in applied mathematics from Xidian University, Xi'an, China, in 2008 and 2013, respectively. He is currently a Professor with Xidian University. He has published more than 30 journal articles with an H-index of 17. His current research interests include evolutionary algorithms and their applications in real world. He was a recipient of the Hong Kong Scholar at the Mainland-Hong Kong Joint Postdoctoral Fellows Program, China, in 2017, and the One-Hundred Talent Program at Shaanxi Province, China, in 2017.